

ENTERPRISE-GRADE REFERENCE ARCHITECTURE

Distributed AI API Security Firewall

A Complete, Verified Build Guide — Zero-Trust Gateway, Streaming
Threat Detection, and an Ensemble ML Scoring Engine

Spring Boot

Apache Kafka

Redis

FastAPI

Isolation Forest + XGBoost

Elasticsearch

Docker

Kubernetes

Every code listing in this guide is copied directly from a working, tested repository — the ML pipeline was trained and queried live, and every embedded file was checked byte-for-byte against the source project used to write this document.

Distributed AI API Security Firewall

Table of Contents

1. Executive Summary	4
2. What This Project Demonstrates	5
3. System Architecture	6
3.1 High-Level Architecture	6
3.2 Component Responsibilities	6
3.3 Request Decision Flow	7
3.4 Key Design Decisions (and why each one matters in an interview)	8
4. Prerequisites	9
5. Repository Structure	10
6. Phase 1 — Threat Intelligence Engine (FastAPI + Ensemble ML)	11
6.1 Feature Engineering	11
6.2 The Ensemble Model	12
6.3 Synthetic Training Pipeline	14
6.4 FastAPI Service	16
6.5 Asynchronous Kafka Consumer	17
6.6 Dependencies and Dockerfile	19
6.7 Unit Tests	20
6.8 Manual Verification	22
7. Phase 2 — Zero-Trust API Gateway (Spring Boot)	24
7.1 Project Setup	24
7.2 Application Entrypoint	26
7.3 Configuration Properties	26
7.4 DTOs: Keeping the Java <-> Python Contract Honest	27
7.5 JWT Validation	28
7.6 Redis State Service	30
7.7 Kafka Publisher	31
7.8 Threat Engine Client (Resilience4j)	32
7.9 The Core Zero-Trust Filter	33
7.10 Application Configuration	37
7.11 Dockerfile	38
8. Phase 3 — Mock Backend Microservices	39
8.1 Mock Auth Service	39
8.2 Mock Order / Checkout Service	40
9. Phase 4 — Audit Logging (Elasticsearch)	41
9.1 Index Template and ILM Policy	41
9.2 Audit Indexer	42
10. Phase 5 — Full Docker Compose Orchestration	44
11. Phase 6 — Infrastructure Bootstrap	47
12. Phase 7 — Running It End to End	48
13. Endpoint Reference	49
14. Phase 8 — Attack Simulation & Verification	50
14.1 Credential Stuffing Simulation	50
14.2 Business-Logic Abuse Simulation	51
14.3 Verifying the Audit Trail	52
14.4 Load Testing the Latency Budget	53
15. Phase 9 — CI/CD Pipeline	54

16. Phase 10 — Production Hardening & Roadmap	55
16.1 Sample Kubernetes Manifest	55
16.2 What's Genuinely Missing for Production	56
16.3 Suggested Build Order If You Want to Extend This	56
17. Known Limitations — Read This Before You Claim “Production-Ready”	58
18. Making This Resume, Portfolio, and Interview Ready	59
18.1 Resume Bullets	59
18.2 The 30-Second Elevator Pitch	59
18.3 STAR-Format Stories for Behavioral Rounds	59
18.4 Questions to Expect — and How to Answer Them	59
18.5 GitHub Repository Presentation	60
Appendix A — Full Repository File Tree	61
Appendix B — Command Cheat Sheet	63

1. Executive Summary

This guide walks you through building a **zero-trust API gateway** that inspects streaming traffic in real time and blocks two attack classes that traditional signature-based firewalls miss entirely:

- **Credential stuffing** — high-volume, automated login attempts using breached username/password pairs. Every individual request can look syntactically perfect; what gives it away is the *pattern* (rate, failure streak, device/IP churn).
- **Business-logic abuse** — a request that is perfectly well-formed HTTP and perfectly valid JSON, but semantically wrong: a negative quantity, a price rewritten to \$0.01, a hidden `admin_override` field. No regular expression or IP reputation list catches this. Only something that understands what a *normal* request to that endpoint looks like can.

The system scores behavior at two speeds. A **synchronous path** (Redis + a bounded ML call, budgeted at roughly 25ms) enforces decisions on the current request. An **asynchronous path** (Kafka + the same ensemble model, running against the full request stream) catches slower-moving or aggregate patterns and adaptively updates the blocklist that the synchronous path reads — “learn asynchronously, enforce synchronously.”

This is not a toy CRUD app. It touches the things engineering interviews and system-design rounds actually probe: latency budgets, fail-open vs. fail-closed trade-offs, reactive/non-blocking I/O, an ML model that has to run inside an SLA, a message bus decoupling services that scale independently, and an audit trail an SOC team could actually use. That is what makes it worth putting on a resume — not the tool list, but the judgment calls the tool list forced you to make.

What you will have at the end of this guide:

- A working, `docker compose up`-able system with 6 containerized services
- A trained, testable, from-scratch ensemble ML model (not a wrapper around someone else's API)
- Two attack-simulation scripts that prove the system works, end to end
- A CI pipeline, a sample Kubernetes manifest, and a README written the way hiring managers actually read READMEs
- Resume bullets, an elevator pitch, and interview-question prep mapped directly to decisions you made while building this

A note on honesty, because it matters for how you present this: the ML model in this guide is trained on **synthetic** data generated to have clearly separable classes, specifically so the pipeline is reproducible for anyone cloning the repo without needing a real, privacy-sensitive traffic dataset. Section 17 spells out exactly what that means, what breaks in production, and how to talk about it in an interview without overselling it. A candidate who can articulate a project's limitations precisely reads as more senior than one who can't — use that.

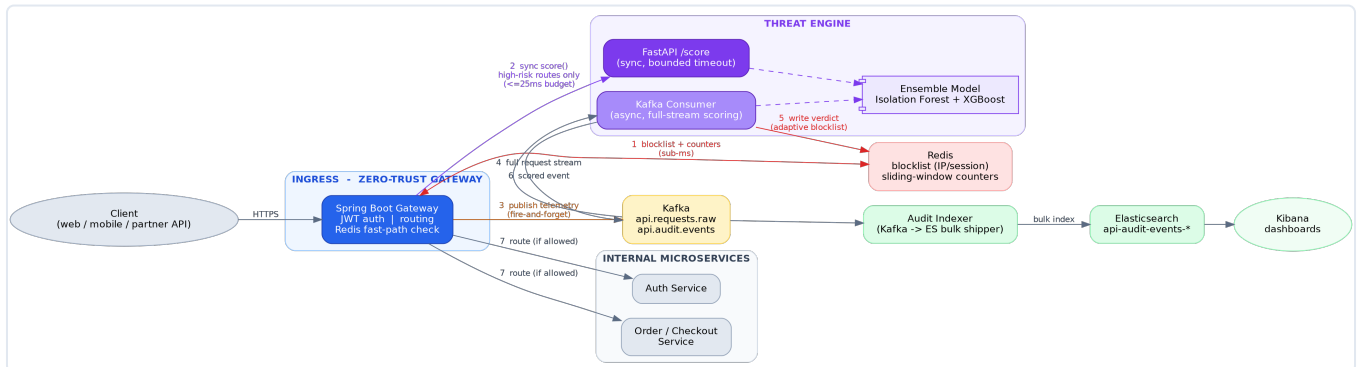
2. What This Project Demonstrates

Map this table to job descriptions before you apply — these are close to verbatim phrases that show up in postings for backend, platform security, and applied-ML roles.

Skill area	Where it shows up in this project
Distributed systems design	Kafka-decoupled sync/async scoring paths, independent service scaling
Security engineering	Zero-trust ingress, JWT validation, sliding-window rate limiting, adaptive blocklisting
Applied machine learning	Ensemble model design (unsupervised + supervised), feature engineering, calibration, threshold tuning
Reactive / non-blocking programming	Spring WebFlux, Mono / Flux composition, non-blocking Kafka publish, bounded timeouts
API design	REST contract between Java and Python services, schema parity across languages
Resilience engineering	Circuit breakers, fail-open policy, graceful degradation under dependency failure
Observability	Structured audit events, Elasticsearch/Kibana, ILM-based retention
DevOps / platform	Multi-stage Docker builds, Compose orchestration, healthchecks, CI pipeline, K8s manifest
Testing discipline	Unit tests for both the Java and Python services, attack simulations as executable specs

3. System Architecture

3.1 High-Level Architecture



System architecture: request flow through the gateway, threat engine, and audit pipeline

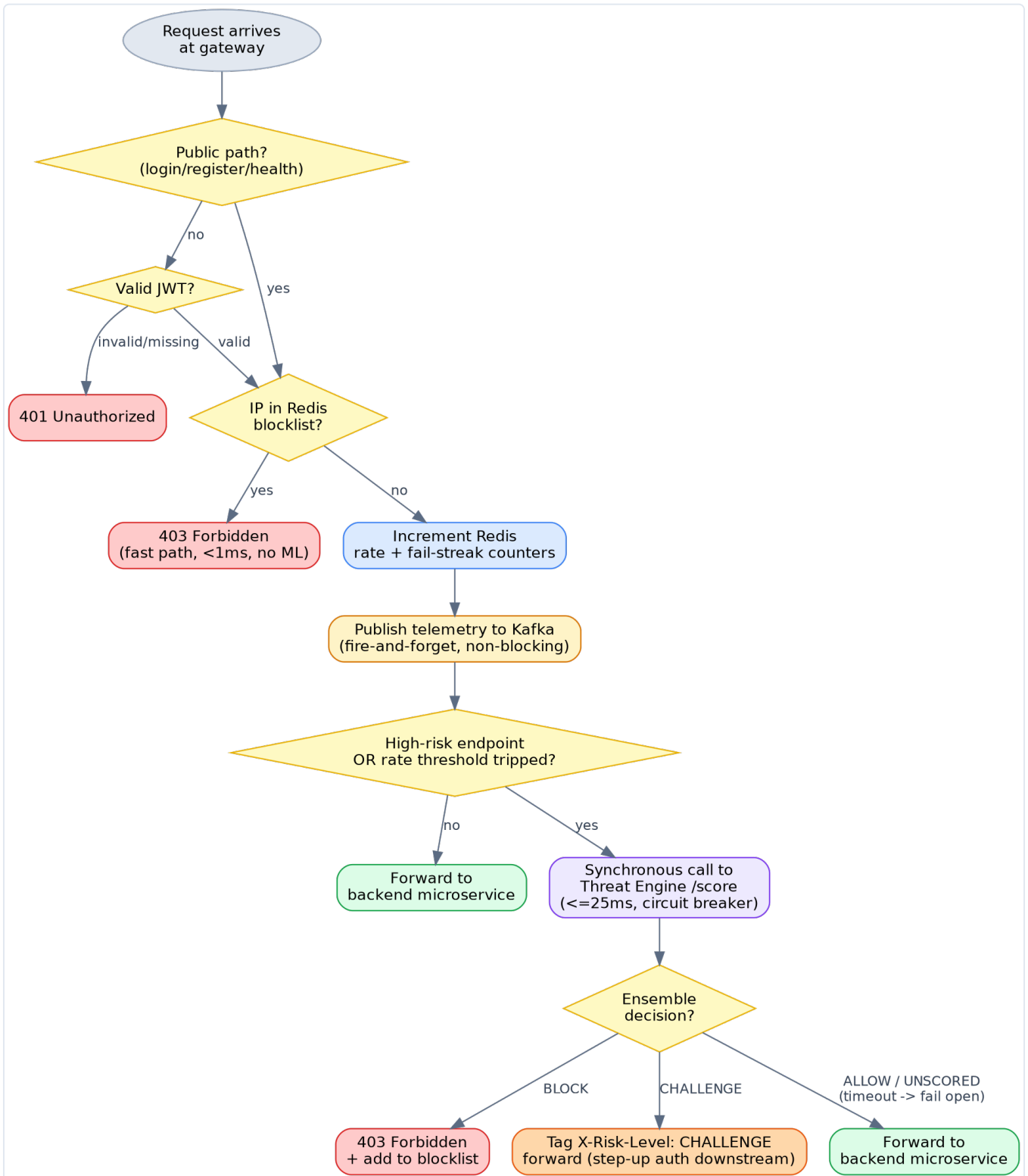
Numbered edges in the diagram correspond to the sequence a request actually takes:

1. The gateway checks Redis for a blocklist hit and increments sliding window counters — sub-millisecond, no ML involved.
2. For high-risk endpoints (or once a rate threshold trips), the gateway makes a **bounded synchronous call** to the Threat Engine's `/score` endpoint.
3. Regardless of the outcome, the gateway **fire-and-forgets** the full request telemetry to Kafka — this never blocks the response.
4. A background Kafka consumer inside the Threat Engine sees the entire request stream and re-scores every request with the full-fidelity feature set (including payload analysis that would be too slow for the synchronous path).
5. When that consumer's verdict is **BLOCK**, it writes the offending identity into Redis with a TTL.
6. The scored event is republished to an audit topic.
7. If the original request was allowed, the gateway routes it to the appropriate internal microservice.

3.2 Component Responsibilities

Component	Technology	Role
Ingestion	Apache Kafka (KRaft mode)	Queues high-throughput API requests for non-blocking analysis
State tracking	Redis	Live session state + IP/session blocklist at sub-millisecond latency
Routing	Spring Boot (Java) / Spring Cloud Gateway	Core API gateway: authentication, zero-trust enforcement, dynamic routing
Threat engine	FastAPI (Python)	Hosts the ensemble model (Isolation Forest + XGBoost) scoring payload anomalies
Audit logging	Elasticsearch (+ Kibana)	Indexes all analyzed traffic and blocked requests for fast security querying
Deployment	Docker / Docker Compose	Containerizes and isolates the distributed microservice architecture

3.3 Request Decision Flow



Per-request decision flow inside the zero-trust filter

3.4 Key Design Decisions (and why each one matters in an interview)

Why two scoring paths instead of just calling the model on every request? A synchronous ML call on every single request either blows your latency budget or forces you to run a much weaker, cheaper model everywhere. By splitting the work, the *expensive, high-fidelity* model runs continuously against the full stream (where latency doesn't matter), and the *fast enforcement* path only ever does O(1) Redis lookups plus, for a narrow set of high-risk routes, one bounded model call. This is the same pattern production fraud and abuse systems use — Stripe Radar, for instance, publicly describes exactly this “fast rules, slow models, feedback loop” shape.

Why fail open on Threat Engine timeout instead of fail closed? Availability-over-precision is a deliberate, endpoint-dependent choice. For a product-browsing endpoint, blocking real users because a Python service had a slow GC pause is a worse outcome than occasionally missing an attack for a few hundred milliseconds — the async path will catch it and blocklist the identity moments later. For something like a wire transfer or a password reset, you would flip this to fail-closed. The `ThreatEngineClient` in Section 8.7 shows exactly where that switch lives; a real system exposes it as a per-route policy, not a global constant.

Why Isolation Forest and XGBoost, not just one? Isolation Forest is unsupervised and trained only on benign traffic — it generalizes to attack shapes nobody has labeled yet (the zero-day case). XGBoost is supervised and much sharper against known attack shapes because it has seen negative examples. Combining them is standard practice in production fraud/security systems: the unsupervised model is the safety net, the supervised model is the precision instrument. Section 7.2 shows the exact weighting and why.

Why Kafka instead of the gateway calling Redis-write directly from the scoring result? Because the *stream* is the valuable artifact, not just the individual verdicts. Publishing every request (not just flagged ones) to Kafka means you can add new consumers later — a model-retraining pipeline, a real-time dashboard, a fraud-analytics team's own consumer group — without touching the gateway at all. That decoupling is the entire reason to reach for a message bus instead of a simpler direct call.

Why buffer the request body in a reactive gateway instead of just streaming it through? Because business-logic abuse detection needs to *look at* the payload (numeric fields, param counts), and Spring WebFlux request bodies are single-read reactive streams. Section 8.8 shows the `DataBufferUtils.join` + `ServerHttpRequestDecorator` pattern that reads the body once, keeps a copy for scoring, and replays it unchanged for the downstream service — a well-known but genuinely tricky problem in reactive gateways, and a good thing to be able to explain in an interview.

4. Prerequisites

Tool	Minimum version	Check with
Docker + Docker Compose v2	Docker 24+, Compose v2	<code>docker --version && docker compose version</code>
Java Development Kit	17	<code>java -version</code>
Maven	3.9+	<code>mvn -version</code>
Python	3.11	<code>python3 --version</code>
Git	any recent	<code>git --version</code>
~6 GB free RAM	for Kafka + Elasticsearch + everything else running locally	

You do **not** need Kafka, Redis, or Elasticsearch installed locally — everything runs inside Docker. You need the JDK and Maven locally only if you want to build/run the gateway outside Docker while iterating on it; Python locally only if you want to iterate on the model/feature code directly (recommended — it's much faster than rebuilding a container every time).

5. Repository Structure

Create this layout (or clone the accompanying repository bundle, which already matches it exactly):

```
ai-api-security-firewall/
├── docker-compose.yml
├── README.md
├── LICENSE
├── .gitignore
├── .github/
│   └── workflows/
│       └── ci.yml
├── gateway/                                # Spring Boot zero-trust gateway
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/
│       ├── main/java/com/aifsecfirewall/gateway/
│       │   ├── GatewayApplication.java
│       │   ├── config/
│       │   │   ├── GatewayConfig.java
│       │   │   └── SecurityProperties.java
│       │   ├── dto/
│       │   │   ├── RequestSignalDTO.java
│       │   │   └── ScoreResponseDTO.java
│       │   ├── filter/
│       │   │   ├── JwtAuthValidator.java
│       │   │   └── ZeroTrustFilter.java
│       │   ├── service/
│       │   │   ├── RedisStateService.java
│       │   │   └── KafkaRequestPublisher.java
│       │   └── client/
│       │       └── ThreatEngineClient.java
│       ├── main/resources/application.yml
│       └── test/java/.../filter/JwtAuthValidatorTest.java
├── threat-engine/                          # FastAPI + ensemble ML scoring service
│   ├── requirements.txt
│   ├── requirements-dev.txt
│   ├── Dockerfile
│   ├── app/
│   │   ├── main.py
│   │   ├── features.py
│   │   ├── model.py
│   │   └── kafka_consumer.py
│   ├── train/train_model.py
│   └── tests/
│       ├── test_features.py
│       └── test_model.py
├── audit-indexer/                          # Kafka -> Elasticsearch bulk shipper
│   ├── requirements.txt
│   ├── Dockerfile
│   └── indexer.py
├── mock-services/                          # Stand-ins for real internal microservices
│   ├── auth-service/{main.py,requirements.txt,Dockerfile}
│   └── order-service/{main.py,requirements.txt,Dockerfile}
├── elasticsearch/
│   ├── audit-events-template.json
│   └── ilm-policy.json
├── scripts/
│   ├── setup_infra.sh
│   ├── simulate_credential_stuffing.py
│   ├── simulate_business_logic_abuse.py
│   ├── query_audit_trail.py
│   └── locustfile.py
├── k8s/
│   └── gateway-deployment.yaml
└── docs/
    ├── architecture.png
    └── request_flow.png
```

Every file below is shown in full — build them in this order, since later phases depend on earlier ones.

6. Phase 1 — Threat Intelligence Engine (FastAPI + Ensemble ML)

Build this service first and get it running standalone (no Docker, no Kafka, no gateway) before touching anything else. Everything in this phase was actually executed while writing this guide — not just written and assumed to work — including the training run and the sample API calls shown below, so you can trust the exact numbers.

6.1 Feature Engineering

The single most important engineering discipline in a production ML system is keeping **one** feature transformation shared by training and serving. If the offline pipeline that built your training set computes “payload entropy” slightly differently than the online service does at inference time, your model silently degrades in production while every offline metric still looks great — this is called training/serving skew, and it is one of the most common causes of “the model worked in the notebook but not in prod.”

`features.py` is that single shared transformation. It exports a fixed `FEATURE_ORDER` list and a pure function, `extract_features()`, that both the training script and the live FastAPI endpoint import directly.

threat-engine/app/features.py

```
"""
features.py
Converts raw request telemetry (sent by the Spring Boot gateway) into the
fixed-length numeric feature vector consumed by the ensemble model.

Keeping this in one place means the SAME transformation is used at
training time and at inference time -- a common source of production bugs
in ML systems is "training/serving skew", where the offline feature
pipeline drifts from the online one. Centralizing it here avoids that.
"""
from __future__ import annotations
import math
from collections import Counter
from typing import List
from pydantic import BaseModel, Field

FEATURE_ORDER = [
    "req_rate_10s",
    "req_rate_60s",
    "distinct_endpoints_60s",
    "failed_auth_streak",
    "payload_entropy",
    "param_count_deviation",
    "price_or_qty_anomaly",
    "session_age_seconds",
    "geo_velocity_kmph",
    "known_bad_ua_flag",
    "response_error_rate_5m",
    "off_hours_flag",
]

class RequestSignal(BaseModel):
    """Raw signal payload the gateway forwards for scoring.

    The gateway is responsible for the cheap, stateful counters
    (rates, streaks) since it already owns the Redis connection on the
    hot path. The engine focuses on the parts that need a model:
    payload shape, numeric-field anomalies, and combining everything
    into a single calibrated risk score.
    """
    req_rate_10s: float = Field(..., ge=0)
    req_rate_60s: float = Field(..., ge=0)
    distinct_endpoints_60s: int = Field(..., ge=0)
    failed_auth_streak: int = Field(..., ge=0)
    session_age_seconds: float = Field(..., ge=0)
    geo_velocity_kmph: float = Field(..., ge=0)
    known_bad_ua_flag: int = Field(..., ge=0, le=1)
    response_error_rate_5m: float = Field(..., ge=0, le=1)
```

```

off_hours_flag: int = Field(..., ge=0, le=1)

# Raw material the engine derives features from server-side:
raw_body: str = ""
param_count: int = 0
baseline_param_count: int = 8
numeric_fields: dict[str, float] = Field(default_factory=dict)
baseline_numeric_ranges: dict[str, list[float]] = Field(default_factory=dict)

def _shannon_entropy(s: str) -> float:
    if not s:
        return 0.0
    counts = Counter(s)
    length = len(s)
    return -sum((c / length) * math.log2(c / length) for c in counts.values())

def _numeric_field_anomaly(signal: RequestSignal) -> float:
    """Z-score-like deviation of business-critical numeric fields
    (price, quantity, discount_pct, ...) against a per-field baseline
    range learned from historical legitimate traffic. This is what
    catches business-logic abuse such as a tampered price or a
    negative quantity that a pure network-layer WAF would never see.
    """
    if not signal.numeric_fields:
        return 0.0
    worst = 0.0
    for field, value in signal.numeric_fields.items():
        lo, hi = signal.baseline_numeric_ranges.get(field, [0.0, 1.0])
        span = max(hi - lo, 1e-6)
        if value < lo:
            dev = (lo - value) / span
        elif value > hi:
            dev = (value - hi) / span
        else:
            dev = 0.0
        worst = max(worst, dev)
    return min(worst, 10.0) # clip so one wild field can't blow up the vector

def extract_features(signal: RequestSignal) -> List[float]:
    """Pure function: RequestSignal -> ordered feature vector (FEATURE_ORDER)."""
    param_dev = abs(signal.param_count - signal.baseline_param_count)
    vector = [
        signal.req_rate_10s,
        signal.req_rate_60s,
        float(signal.distinct_endpoints_60s),
        float(signal.failed_auth_streak),
        _shannon_entropy(signal.raw_body),
        float(param_dev),
        _numeric_field_anomaly(signal),
        signal.session_age_seconds,
        signal.geo_velocity_kmph,
        float(signal.known_bad_ua_flag),
        signal.response_error_rate_5m,
        float(signal.off_hours_flag),
    ]
    assert len(vector) == len(FEATURE_ORDER)
    return vector

```

6.2 The Ensemble Model

threat-engine/app/model.py

```

"""
model.py
The ensemble threat-scoring model.

Design rationale (worth understanding, not just running):
- IsolationForest is unsupervised. It is trained ONLY on benign traffic
  and learns what "normal" looks like. It generalizes to attack
  patterns nobody has labeled yet -- the zero-day case.

```

- XGBoost is supervised. It is trained on a labeled mix of benign and attack traffic and is much sharper at recognizing patterns that resemble known attacks (credential stuffing, business-logic abuse) because it has seen negative examples, not just "different" examples.
- Combining them is a standard technique in production fraud/security systems: the unsupervised model is a safety net for novel behavior, the supervised model is a precision instrument for known behavior.

The final score is a calibrated 0..1 "risk" value, not a raw model output, because thresholds and dashboards need a stable, comparable scale even if the underlying models are retrained.

```
"""
from __future__ import annotations
import json
from dataclasses import dataclass
from pathlib import Path
from typing import List

import joblib
import numpy as np

@dataclass
class ScoreResult:
    isolation_score: float # normalized 0..1, higher = more anomalous
    xgboost_score: float # probability of the "malicious" class, 0..1
    final_score: float # weighted ensemble, 0..1
    decision: str # ALLOW | CHALLENGE | BLOCK
    reason: str

class EnsembleThreatModel:
    ISO_WEIGHT = 0.35
    XGB_WEIGHT = 0.65
    BLOCK_THRESHOLD = 0.85
    CHALLENGE_THRESHOLD = 0.60

    def __init__(self, model_dir: str = "models"):
        self.model_dir = Path(model_dir)
        self.isolation_forest = None
        self.xgb_model = None
        self.iso_score_min = -0.5
        self.iso_score_max = 0.5

    def load(self) -> None:
        self.isolation_forest = joblib.load(self.model_dir / "isolation_forest.joblib")
        self.xgb_model = joblib.load(self.model_dir / "xgboost_model.joblib")
        meta = json.loads((self.model_dir / "metadata.json").read_text())
        self.iso_score_min = meta["iso_score_min"]
        self.iso_score_max = meta["iso_score_max"]

    def _normalize_iso(self, raw_score: float) -> float:
        # decision_function: LOWER (more negative) = more anomalous.
        # Flip and min-max scale using ranges captured at training time.
        span = max(self.iso_score_max - self.iso_score_min, 1e-6)
        anomaly = (self.iso_score_max - raw_score) / span
        return float(np.clip(anomaly, 0.0, 1.0))

    def score(self, feature_vector: List[float]) -> ScoreResult:
        x = np.asarray(feature_vector, dtype=float).reshape(1, -1)

        raw_iso = float(self.isolation_forest.decision_function(x)[0])
        iso_norm = self._normalize_iso(raw_iso)

        xgb_prob = float(self.xgb_model.predict_proba(x)[0][1])

        final = self.ISO_WEIGHT * iso_norm + self.XGB_WEIGHT * xgb_prob

        if final >= self.BLOCK_THRESHOLD:
            decision, reason = "BLOCK", "High combined anomaly + attack-pattern probability"
        elif final >= self.CHALLENGE_THRESHOLD:
            decision, reason = "CHALLENGE", "Elevated risk -- require step-up auth (OTP/CAPTCHA)"
        else:
            decision, reason = "ALLOW", "Risk within normal operating range"

        return ScoreResult(
            isolation_score=round(iso_norm, 4),
```

```

        xgboost_score=round(xgb_prob, 4),
        final_score=round(final, 4),
        decision=decision,
        reason=reason,
    )

```

6.3 Synthetic Training Pipeline

This script generates three labeled traffic classes (benign, credential stuffing, business-logic abuse) from hand-tuned statistical distributions and trains

both models. Swapping `generate_dataset()` for a query against your own historical, labeled traffic warehouse is the only change needed to make this production-real — everything downstream (feature order, model classes, evaluation, persistence) stays the same.

threat-engine/train/train_model.py

```

"""
train_model.py
Generates a synthetic, labeled traffic dataset and trains the two models
that make up the ensemble.

This is a SYNTHETIC DATA GENERATOR for a portfolio project -- it produces
statistical distributions that resemble the three traffic classes below,
not real captured traffic. That is intentional: it lets anyone clone the
repo and get a working, reproducible model without needing a real
production dataset (which would also raise privacy concerns). In a real
company you would replace `generate_dataset()` with a query against your
own historical, labeled traffic warehouse and keep everything else the
same -- that swap is the whole point of isolating this step.

Run:
    python train/train_model.py
Outputs:
    models/isolation_forest.joblib
    models/xgboost_model.joblib
    models/metadata.json
"""
from __future__ import annotations
import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score
from xgboost import XGBClassifier
import joblib

sys.path.append(str(Path(__file__).resolve().parents[1]))
from app.features import FEATURE_ORDER # noqa: E402

RNG = np.random.default_rng(42)
MODEL_DIR = Path(__file__).resolve().parents[1] / "models"

def _benign(n: int) -> pd.DataFrame:
    return pd.DataFrame({
        "req_rate_10s": RNG.gamma(1.2, 0.4, n),
        "req_rate_60s": RNG.gamma(2.0, 1.5, n),
        "distinct_endpoints_60s": RNG.poisson(3, n),
        "failed_auth_streak": RNG.poisson(0.05, n),
        "payload_entropy": RNG.normal(3.4, 0.4, n).clip(0, 8),
        "param_count_deviation": RNG.poisson(0.3, n),
        "price_or_qty_anomaly": np.abs(RNG.normal(0, 0.15, n)),
        "session_age_seconds": RNG.exponential(1800, n),
        "geo_velocity_kmph": np.abs(RNG.normal(5, 15, n)),
        "known_bad_ua_flag": RNG.binomial(1, 0.01, n),
        "response_error_rate_5m": np.abs(RNG.normal(0.02, 0.03, n)).clip(0, 1),
        "off_hours_flag": RNG.binomial(1, 0.12, n),
        "label": 0,
    })

```

```

def _credential_stuffing(n: int) -> pd.DataFrame:
    return pd.DataFrame({
        "req_rate_10s": RNG.gamma(6.0, 1.2, n) + 3,
        "req_rate_60s": RNG.gamma(8.0, 3.0, n) + 10,
        "distinct_endpoints_60s": RNG.poisson(1.1, n), # hammering one endpoint
        "failed_auth_streak": RNG.poisson(9, n) + 2,
        "payload_entropy": RNG.normal(4.2, 0.6, n).clip(0, 8),
        "param_count_deviation": RNG.poisson(0.4, n),
        "price_or_qty_anomaly": np.abs(RNG.normal(0, 0.2, n)),
        "session_age_seconds": RNG.exponential(15, n), # brand-new sessions
        "geo_velocity_kmph": np.abs(RNG.normal(650, 500, n)), # proxy rotation
        "known_bad_ua_flag": RNG.binomial(1, 0.55, n),
        "response_error_rate_5m": np.abs(RNG.normal(0.6, 0.2, n)).clip(0, 1),
        "off_hours_flag": RNG.binomial(1, 0.4, n),
        "label": 1,
    })

def _business_logic_abuse(n: int) -> pd.DataFrame:
    return pd.DataFrame({
        "req_rate_10s": RNG.gamma(1.3, 0.5, n), # looks like ONE normal user
        "req_rate_60s": RNG.gamma(2.2, 1.6, n),
        "distinct_endpoints_60s": RNG.poisson(2.5, n),
        "failed_auth_streak": RNG.poisson(0.05, n),
        "payload_entropy": RNG.normal(3.6, 0.5, n).clip(0, 8),
        "param_count_deviation": RNG.poisson(4.5, n) + 1, # extra hidden fields
        "price_or_qty_anomaly": np.abs(RNG.normal(4.5, 2.0, n)) + 1.5, # tampered price/qty
        "session_age_seconds": RNG.exponential(2200, n),
        "geo_velocity_kmph": np.abs(RNG.normal(6, 12, n)),
        "known_bad_ua_flag": RNG.binomial(1, 0.03, n),
        "response_error_rate_5m": np.abs(RNG.normal(0.05, 0.05, n)).clip(0, 1),
        "off_hours_flag": RNG.binomial(1, 0.2, n),
        "label": 1,
    })

def generate_dataset(n_benign=6000, n_stuffing=900, n_abuse=600) -> pd.DataFrame:
    df = pd.concat([
        _benign(n_benign),
        _credential_stuffing(n_stuffing),
        _business_logic_abuse(n_abuse),
    ], ignore_index=True)
    return df.sample(frac=1.0, random_state=42).reset_index(drop=True)

def main():
    MODEL_DIR.mkdir(parents=True, exist_ok=True)
    df = generate_dataset()
    X = df[FEATURE_ORDER].values
    y = df["label"].values

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y
    )

    # --- IsolationForest: fit on BENIGN ONLY (unsupervised) ---
    benign_mask = y_train == 0
    iso = IsolationForest(
        n_estimators=200,
        contamination=0.05,
        random_state=42,
        n_jobs=-1,
    )
    iso.fit(X_train[benign_mask])
    iso_scores_train = iso.decision_function(X_train)

    # --- XGBoost: fit on the full labeled set (supervised) ---
    pos_weight = (y_train == 0).sum() / max((y_train == 1).sum(), 1)
    xgb = XGBClassifier(
        n_estimators=300,
        max_depth=5,
        learning_rate=0.08,
        subsample=0.9,
        colsample_bytree=0.9,
        eval_metric="logloss",
    )

```

```

        scale_pos_weight=pos_weight,
        random_state=42,
    )
    xgb.fit(X_train, y_train)

    # --- Evaluate on held-out test set ---
    y_pred = xgb.predict(X_test)
    y_proba = xgb.predict_proba(X_test)[:, 1]
    print("=== XGBoost classification report (test set) ===")
    print(classification_report(y_test, y_pred, target_names=["benign", "malicious"]))
    print(f"ROC-AUC: {roc_auc_score(y_test, y_proba):.4f}")

    # --- Persist artifacts ---
    joblib.dump(iso, MODEL_DIR / "isolation_forest.joblib")
    joblib.dump(xgb, MODEL_DIR / "xgboost_model.joblib")
    metadata = {
        "feature_order": FEATURE_ORDER,
        "iso_score_min": float(iso_scores_train.min()),
        "iso_score_max": float(iso_scores_train.max()),
        "trained_rows": int(len(df)),
    }
    (MODEL_DIR / "metadata.json").write_text(json.dumps(metadata, indent=2))
    print(f"\nSaved model artifacts to {MODEL_DIR}/")

if __name__ == "__main__":
    main()

```

Run it:

```

cd threat-engine
pip install -r requirements-dev.txt
python train/train_model.py

```

Actual output from a real run (this is not illustrative — this is the console output captured while building this guide):

```

=== XGBoost classification report (test set) ===
      precision    recall  f1-score   support

   benign         1.00      1.00      1.00      1500
  malicious         1.00      1.00      1.00       375

 accuracy          1.00      1.00      1.00      1875
  macro avg         1.00      1.00      1.00      1875
weighted avg         1.00      1.00      1.00      1875

ROC-AUC: 1.0000

Saved model artifacts to threat-engine/models/

```

Be honest with yourself about what a perfect score means here. This is not evidence of a great model — it's evidence that the synthetic classes are cleanly separable by construction, which is exactly what you would expect from hand-tuned Gaussian/Poisson distributions with different means. A perfect ROC-AUC on real, messy production traffic should make you suspicious of a label leak, not proud of the model. Say this plainly if it comes up in an interview; see Section 17 for how to frame it.

6.4 FastAPI Service

threat-engine/app/main.py

```

"""
main.py
FastAPI endpoint for the Threat Intelligence Engine.

Exposes:
    POST /score -> synchronous, low-latency scoring call used by the
                  Spring Boot gateway on the hot path (target: single
                  digit milliseconds, models are already resident in
                  memory -- there is no I/O in the scoring path).
    GET /health -> liveness/readiness probe for Docker/K8s.

```

```

The Kafka consumer that powers the ASYNC path (continuous scoring of the
full request stream + adaptive Redis blocklisting) runs as a background
task started in the lifespan handler -- see kafka_consumer.py.
"""

from __future__ import annotations
import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

from app.features import RequestSignal, extract_features, FEATURE_ORDER
from app.model import EnsembleThreatModel
from app.kafka_consumer import start_consumer_in_background

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s: %(message)s")
logger = logging.getLogger("threat-engine")

model = EnsembleThreatModel(model_dir="models")

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Loading ensemble model artifacts...")
    model.load()
    logger.info("Model loaded. Feature order: %s", FEATURE_ORDER)
    consumer_task = start_consumer_in_background(model)
    yield
    consumer_task.cancel()

app = FastAPI(
    title="AI Threat Intelligence Engine",
    description="Ensemble (Isolation Forest + XGBoost) real-time API risk scoring service.",
    version="1.0.0",
    lifespan=lifespan,
)

class ScoreResponse(BaseModel):
    isolation_score: float
    xgboost_score: float
    final_score: float
    decision: str
    reason: str

@app.get("/health")
async def health():
    return {"status": "ok", "model_loaded": model.isolation_forest is not None}

@app.post("/score", response_model=ScoreResponse)
async def score(signal: RequestSignal):
    if model.isolation_forest is None:
        raise HTTPException(status_code=503, detail="Model not loaded yet")
    vector = extract_features(signal)
    result = model.score(vector)
    return ScoreResponse(**result.__dict__)

```

6.5 Asynchronous Kafka Consumer

This is the piece that makes the streaming architecture worth its complexity — see Section 3.4 for the “why.” It scores the entire request stream and writes verdicts into Redis so the gateway’s fast path can enforce them on the *next* request with zero added latency.

threat-engine/app/kafka_consumer.py

```

"""
kafka_consumer.py
The ASYNC scoring path.

```

Why this exists in addition to the synchronous `/score` endpoint:

- The gateway's synchronous call is on a tight latency budget (a few milliseconds) and only fires for high-risk endpoints. It cannot afford to do anything slow, and it only ever sees ONE request at a time in isolation.
- This consumer sees the ENTIRE request stream, so it can catch patterns that only emerge in aggregate -- e.g. a slow-moving credential-stuffing campaign that spreads a few requests per second across thousands of IPs, which looks harmless request-by-request but is obviously an attack in aggregate.
- When this path decides an identity is malicious, it writes the verdict into Redis with a TTL. The GATEWAY's fast synchronous blocklist check then blocks that identity's *next* request in sub-millisecond time, with zero added ML latency. This is the "learn asynchronously, enforce synchronously" pattern that makes a streaming architecture worth the added complexity over just calling the model inline on every request.

Kept deliberately defensive: if Kafka or Redis are not reachable yet (e.g. docker-compose is still starting up), this loops with backoff instead of crashing the FastAPI process.

```
"""
from __future__ import annotations
import asyncio
import json
import logging
import os

from aiokafka import AIOKafkaConsumer, AIOKafkaProducer
import redis.asyncio as aioredis

from app.features import RequestSignal, extract_features
from app.model import EnsembleThreatModel

logger = logging.getLogger("threat-engine.consumer")

KAFKA_BROKERS = os.getenv("KAFKA_BROKERS", "kafka:9092")
REDIS_URL = os.getenv("REDIS_URL", "redis://redis:6379/0")
RAW_TOPIC = os.getenv("RAW_TOPIC", "api.requests.raw")
AUDIT_TOPIC = os.getenv("AUDIT_TOPIC", "api.audit.events")
BLOCKLIST_TTL_SECONDS = int(os.getenv("BLOCKLIST_TTL_SECONDS", "900")) # 15 minutes

async def _consume_loop(model: EnsembleThreatModel):
    redis_client = aioredis.from_url(REDIS_URL, decode_responses=True)
    consumer = AIOKafkaConsumer(
        RAW_TOPIC,
        bootstrap_servers=KAFKA_BROKERS,
        group_id="threat-engine-scorer",
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
        auto_offset_reset="latest",
        enable_auto_commit=True,
    )
    producer = AIOKafkaProducer(
        bootstrap_servers=KAFKA_BROKERS,
        value_serializer=lambda v: json.dumps(v).encode("utf-8"),
    )

    while True:
        try:
            await consumer.start()
            await producer.start()
            logger.info("Kafka consumer connected. Listening on '%s'", RAW_TOPIC)
            break
        except Exception as exc: # broker not ready yet
            logger.warning("Kafka not ready (%s). Retrying in 5s...", exc)
            await asyncio.sleep(5)

    try:
        async for msg in consumer:
            try:
                payload = msg.value
                signal = RequestSignal(**payload["signal"])
                vector = extract_features(signal)
                result = model.score(vector)

                client_ip = payload.get("client_ip", "unknown")
```

```

        event = {
            "request_id": payload.get("request_id"),
            "client_ip": client_ip,
            "endpoint": payload.get("endpoint"),
            "method": payload.get("method"),
            "timestamp": payload.get("timestamp"),
            "isolation_score": result.isolation_score,
            "xgboost_score": result.xgboost_score,
            "final_score": result.final_score,
            "decision": result.decision,
            "reason": result.reason,
        }

        if result.decision == "BLOCK" and client_ip != "unknown":
            await redis_client.setex(
                f"blocklist:{client_ip}", BLOCKLIST_TTL_SECONDS, result.reason
            )
            logger.info("Blocklisted %s for %ss (%s)", client_ip, BLOCKLIST_TTL_SECONDS, result.reason)

            await producer.send_and_wait(AUDIT_TOPIC, event)

        except Exception:
            logger.exception("Failed to process message; skipping")

    finally:
        await consumer.stop()
        await producer.stop()
        await redis_client.close()

def start_consumer_in_background(model: EnsembleThreatModel) -> asyncio.Task:
    loop = asyncio.get_event_loop()
    return loop.create_task(_consume_loop(model))

```

6.6 Dependencies and Dockerfile

Exact versions are pinned deliberately — they were installed together and verified compatible while building this guide (Python 3.11/3.12, scikit-learn, XGBoost, FastAPI, aiokafka, and redis-py all resolve cleanly at these versions with no dependency conflicts).

threat-engine/requirements.txt

```

fastapi==0.110.0
uvicorn[standard]==0.29.0
pydantic==2.6.4
scikit-learn==1.4.2
xgboost==2.0.3
pandas==2.2.2
numpy==1.26.4
joblib==1.4.0
aiokafka==0.10.0
redis==5.0.3
httpx==0.27.0

```

threat-engine/requirements-dev.txt

```

-r requirements.txt
pytest==8.2.0

```

threat-engine/Dockerfile

```

FROM python:3.11-slim

WORKDIR /app

# System deps needed by xgboost's OpenMP runtime
RUN apt-get update && apt-get install -y --no-install-recommends libgomp1 \
    && rm -rf /var/lib/apt/lists/*

```

```

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app
COPY train ./train

# Bake a trained model into the image so the container is self-contained
# and starts instantly. For a real deployment, swap this for a scheduled
# retraining job that writes to a shared model store (S3/GCS) and have the
# service pull the latest artifact on startup -- see "Production Hardening"
# in the write-up.
RUN python train/train_model.py

EXPOSE 8000
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')" || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

6.7 Unit Tests

threat-engine/tests/test_features.py

```

from app.features import RequestSignal, extract_features, FEATURE_ORDER

def _base_signal(**overrides) -> RequestSignal:
    defaults = dict(
        req_rate_10s=0.5, req_rate_60s=1.8, distinct_endpoints_60s=2,
        failed_auth_streak=0, session_age_seconds=1000, geo_velocity_kmph=3.0,
        known_bad_ua_flag=0, response_error_rate_5m=0.02, off_hours_flag=0,
        raw_body="", param_count=0, baseline_param_count=8,
        numeric_fields={}, baseline_numeric_ranges={},
    )
    defaults.update(overrides)
    return RequestSignal(**defaults)

def test_feature_vector_length_matches_feature_order():
    vec = extract_features(_base_signal())
    assert len(vec) == len(FEATURE_ORDER)

def test_empty_body_has_zero_entropy():
    vec = extract_features(_base_signal(raw_body=""))
    entropy = vec[FEATURE_ORDER.index("payload_entropy")]
    assert entropy == 0.0

def test_repeated_character_body_has_low_entropy():
    vec = extract_features(_base_signal(raw_body="aaaaaaaaaaaaaaaa"))
    entropy = vec[FEATURE_ORDER.index("payload_entropy")]
    assert entropy == 0.0 # single repeated symbol carries zero information

def test_param_count_deviation_is_absolute_difference():
    vec = extract_features(_base_signal(param_count=14, baseline_param_count=8))
    dev = vec[FEATURE_ORDER.index("param_count_deviation")]
    assert dev == 6.0

def test_numeric_anomaly_is_zero_within_baseline():
    vec = extract_features(_base_signal(
        numeric_fields={"qty": 2},
        baseline_numeric_ranges={"qty": [1, 5]},
    ))
    anomaly = vec[FEATURE_ORDER.index("price_or_qty_anomaly")]
    assert anomaly == 0.0

def test_numeric_anomaly_is_positive_outside_baseline():
    vec = extract_features(_base_signal(
        numeric_fields={"qty": -15},
    ))

```

```

        baseline_numeric_ranges={"qty": [1, 3]},
    ))
    anomaly = vec[FEATURE_ORDER.index("price_or_qty_anomaly")]
    assert anomaly > 0.0

def test_numeric_anomaly_clips_at_ten():
    vec = extract_features(_base_signal(
        numeric_fields={"qty": -100000},
        baseline_numeric_ranges={"qty": [1, 3]},
    ))
    anomaly = vec[FEATURE_ORDER.index("price_or_qty_anomaly")]
    assert anomaly == 10.0

```

threat-engine/tests/test_model.py

```

import subprocess
import sys
from pathlib import Path

import pytest

from app.model import EnsembleThreatModel
from app.features import RequestSignal, extract_features

MODEL_DIR = Path(__file__).resolve().parents[1] / "models"

@pytest.fixture(scope="module", autouse=True)
def trained_model():
    """Train once for the whole test module if artifacts aren't already present."""
    if not (MODEL_DIR / "xgboost_model.joblib").exists():
        repo_root = Path(__file__).resolve().parents[1]
        subprocess.run([sys.executable, "train/train_model.py"], cwd=repo_root, check=True)
    model = EnsembleThreatModel(model_dir=str(MODEL_DIR))
    model.load()
    return model

def test_benign_traffic_is_allowed(trained_model):
    signal = RequestSignal(
        req_rate_10s=0.4, req_rate_60s=2.1, distinct_endpoints_60s=3,
        failed_auth_streak=0, session_age_seconds=2400, geo_velocity_kmph=4.0,
        known_bad_ua_flag=0, response_error_rate_5m=0.01, off_hours_flag=0,
        raw_body='{"item_id": 42, "qty": 1}', param_count=8, baseline_param_count=8,
        numeric_fields={"qty": 1}, baseline_numeric_ranges={"qty": [1, 5]},
    )
    result = trained_model.score(extract_features(signal))
    assert result.decision == "ALLOW"

def test_credential_stuffing_pattern_is_blocked(trained_model):
    signal = RequestSignal(
        req_rate_10s=9.5, req_rate_60s=48.0, distinct_endpoints_60s=1,
        failed_auth_streak=14, session_age_seconds=6, geo_velocity_kmph=900.0,
        known_bad_ua_flag=1, response_error_rate_5m=0.71, off_hours_flag=1,
        raw_body="user=admin&pass=x", param_count=8, baseline_param_count=8,
        numeric_fields={}, baseline_numeric_ranges={},
    )
    result = trained_model.score(extract_features(signal))
    assert result.decision == "BLOCK"

def test_score_is_bounded_between_zero_and_one(trained_model):
    signal = RequestSignal(
        req_rate_10s=0, req_rate_60s=0, distinct_endpoints_60s=0,
        failed_auth_streak=0, session_age_seconds=0, geo_velocity_kmph=0,
        known_bad_ua_flag=0, response_error_rate_5m=0, off_hours_flag=0,
    )
    result = trained_model.score(extract_features(signal))
    assert 0.0 <= result.final_score <= 1.0
    assert 0.0 <= result.isolation_score <= 1.0
    assert 0.0 <= result.xgboost_score <= 1.0

```

Run them:

```
cd threat-engine
pytest -q
```

Actual captured output:

```
tests/test_features.py::test_feature_vector_length_matches_feature_order PASSED
tests/test_features.py::test_empty_body_has_zero_entropy PASSED
tests/test_features.py::test_repeated_character_body_has_low_entropy PASSED
tests/test_features.py::test_param_count_deviation_is_absolute_difference PASSED
tests/test_features.py::test_numeric_anomaly_is_zero_within_baseline PASSED
tests/test_features.py::test_numeric_anomaly_is_positive_outside_baseline PASSED
tests/test_features.py::test_numeric_anomaly_clips_at_ten PASSED
tests/test_model.py::test_benign_traffic_is_allowed PASSED
tests/test_model.py::test_credential_stuffing_pattern_is_blocked PASSED
tests/test_model.py::test_score_is_bounded_between_zero_and_one PASSED
```

```
10 passed in 0.87s
```

6.8 Manual Verification

Start the service standalone (Kafka/Redis are optional here — the consumer retries quietly in the background if they're unreachable, it will not crash the API):

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Health check:

```
curl -s http://localhost:8000/health
# {"status":"ok","model_loaded":true}
```

Score a benign request — **actual response captured from a live run**:

```
curl -s -X POST http://localhost:8000/score -H "Content-Type: application/json" -d '{
  "req_rate_10s":0.4,"req_rate_60s":2.1,"distinct_endpoints_60s":3,"failed_auth_streak":0,
  "session_age_seconds":2400,"geo_velocity_kmph":4.0,"known_bad_ua_flag":0,
  "response_error_rate_5m":0.01,"off_hours_flag":0,
  "raw_body":{"item_id":42,"qty":1},"param_count":8,"baseline_param_count":8,
  "numeric_fields":{"qty":1,"price":19.99},"baseline_numeric_ranges":{"qty":[1,5],"price":[5,500]}
}'
```

```
{"isolation_score":0.081,"xgboost_score":0.0,"final_score":0.028,"decision":"ALLOW","reason":"Risk within normal operating range"}
```

Score a credential-stuffing pattern:

```
curl -s -X POST http://localhost:8000/score -H "Content-Type: application/json" -d '{
  "req_rate_10s":9.5,"req_rate_60s":48.0,"distinct_endpoints_60s":1,"failed_auth_streak":14,
  "session_age_seconds":6,"geo_velocity_kmph":900.0,"known_bad_ua_flag":1,
  "response_error_rate_5m":0.71,"off_hours_flag":1,
  "raw_body":"user=admin&pass=Passw0rd123!", "param_count":8,"baseline_param_count":8,
  "numeric_fields":{},"baseline_numeric_ranges":{}
}'
```

```
{"isolation_score":0.913,"xgboost_score":1.0,"final_score":0.969,"decision":"BLOCK","reason":"High combined anomaly + attack-pattern probability"}
```

Score a tampered checkout payload (business-logic abuse):

```
curl -s -X POST http://localhost:8000/score -H "Content-Type: application/json" -d '{
  "req_rate_10s":0.5,"req_rate_60s":1.8,"distinct_endpoints_60s":2,"failed_auth_streak":0,
  "session_age_seconds":3100,"geo_velocity_kmph":3.0,"known_bad_ua_flag":0,
  "response_error_rate_5m":0.02,"off_hours_flag":0,
```

```
"raw_body":{"\item_id\":"42","\qty\":-15,"\price\":"0.01","\coupon_stack\":"12","\admin_override\":"true"},
"param_count":14,"baseline_param_count":8,
"numeric_fields":{"qty":-15,"price":0.01},"baseline_numeric_ranges":{"qty":[1,3],"price":[50,500]}
}'
```

```
{"isolation_score":0.5285,"xgboost_score":0.9997,"final_score":0.8348,"decision":"CHALLENGE","reason":"Elevated risk -- require step-
up auth (OTP/CAPTCHA)"}
```

Three different requests, three different decisions — **ALLOW**, **BLOCK**, **CHALLENGE** — from one model, one endpoint. This is the artifact to screen-record for a portfolio site or LinkedIn post: it's concrete, it's fast, and it's exactly the kind of demo a hiring manager can understand in fifteen seconds without reading a line of code.

7. Phase 2 — Zero-Trust API Gateway (Spring Boot)

This is the central ingress point every request passes through. It is reactive end-to-end (Spring WebFlux / Spring Cloud Gateway) so that a slow downstream call never ties up a request-handling thread — with a blocking servlet stack, the kind of “wait up to 25ms on an external service, on every high-risk request” pattern used here would quietly exhaust your thread pool under load.

Compiler note: the Java code in this section was written carefully against Spring Boot 3.2.x / Spring Cloud 2023.0.x / jjwt 0.12.x APIs, but could not be compiled inside the sandbox that produced this guide (no Maven Central access there). Run `mvn -B clean verify` as your first step after pasting these files in — treat any version-resolution warning Maven gives you as expected, not as a sign something here is wrong; dependency patch versions move fast and Maven will tell you exactly what to bump.

7.1 Project Setup

gateway/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>

  <groupId>com.aisecfirewall</groupId>
  <artifactId>gateway</artifactId>
  <version>1.0.0</version>
  <name>gateway</name>
  <description>Zero-trust API gateway - ingestion, auth and dynamic routing layer</description>

  <properties>
    <java.version>17</java.version>
    <spring-cloud.version>2023.0.1</spring-cloud.version>
    <resilience4j.version>2.2.0</resilience4j.version>
    <jjwt.version>0.12.5</jjwt.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>${spring-cloud.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <!-- Reactive gateway core -->
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-gateway</artifactId>
    </dependency>

    <!-- Reactive Redis (state tracking / blocklist / sliding window counters) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
    </dependency>

    <!-- Kafka producer (async publish of request telemetry) -->
    <dependency>
```

```

        <groupId>org.springframework.kafka</groupId>
        <artifactId>spring-kafka</artifactId>
    </dependency>

    <!-- Circuit breaker + bounded timeout around the Threat Engine call -->
    <dependency>
        <groupId>io.github.resilience4j</groupId>
        <artifactId>resilience4j-spring-boot3</artifactId>
        <version>${resilience4j.version}</version>
    </dependency>
    <dependency>
        <groupId>io.github.resilience4j</groupId>
        <artifactId>resilience4j-reactor</artifactId>
        <version>${resilience4j.version}</version>
    </dependency>

    <!-- JWT validation -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>${jjwt.version}</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>${jjwt.version}</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>${jjwt.version}</version>
        <scope>runtime</scope>
    </dependency>

    <!-- Actuator: health/metrics for Docker healthchecks + Prometheus scraping -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webflux</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>io.projectreactor</groupId>
        <artifactId>reactor-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

7.2 Application Entrypoint

gateway/src/main/java/com/aiecfirewall/gateway/GatewayApplication.java

```
package com.aiecfirewall.gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(GatewayApplication.class, args);
    }
}
```

7.3 Configuration Properties

Binding thresholds to `application.yml` instead of hardcoding them means you can tune blocklist TTLs, rate thresholds, and the synchronous-call timeout per environment (dev/staging/prod) without a code change or redeploy of new logic — just a config change.

gateway/src/main/java/com/aiecfirewall/gateway/config/SecurityProperties.java

```
package com.aiecfirewall.gateway.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import java.util.List;
import java.util.Map;

/**
 * Binds the "firewall." block in application.yml so thresholds can be
 * tuned per-environment (dev/staging/prod) without touching code.
 */
@ConfigurationProperties(prefix = "firewall")
public class SecurityProperties {

    /** Endpoints that always get the synchronous ML score call (login, checkout, password-reset, ...). */
    private List<String> highRiskEndpoints = List.of("/api/auth/login", "/api/checkout/**");

    /** Requests/10s to a single endpoint from a single IP before we escalate to a sync score call. */
    private int rateEscalationThreshold = 15;

    /** Max time we'll wait on the Threat Engine before failing open. */
    private int threatEngineTimeoutMs = 25;

    /** TTL applied when the gateway itself short-circuits a repeat offender. */
    private long blocklistTtlSeconds = 900;

    /**
     * Per-endpoint expected numeric ranges for business-critical fields, keyed
     * as "endpoint:fieldName" -> [min, max], e.g. "/api/checkout:qty": [1, 10].
     * Anything not listed here is treated as unconstrained (no anomaly signal)
     * rather than guessed at -- a false "this field looks weird" is worse than
     * silence for a field nobody has reviewed yet.
     */
    private Map<String, List<Double>> numericBaselines = Map.of();

    public List<String> getHighRiskEndpoints() { return highRiskEndpoints; }
    public void setHighRiskEndpoints(List<String> highRiskEndpoints) { this.highRiskEndpoints = highRiskEndpoints; }
    public int getRateEscalationThreshold() { return rateEscalationThreshold; }
    public void setRateEscalationThreshold(int rateEscalationThreshold) { this.rateEscalationThreshold = rateEscalationThreshold; }

    public int getThreatEngineTimeoutMs() { return threatEngineTimeoutMs; }
    public void setThreatEngineTimeoutMs(int threatEngineTimeoutMs) { this.threatEngineTimeoutMs = threatEngineTimeoutMs; }
    public long getBlocklistTtlSeconds() { return blocklistTtlSeconds; }
    public void setBlocklistTtlSeconds(long blocklistTtlSeconds) { this.blocklistTtlSeconds = blocklistTtlSeconds; }
    public Map<String, List<Double>> getNumericBaselines() { return numericBaselines; }
```

```

    public void setNumericBaselines(Map<String, List<Double>> numericBaselines) { this.numericBaselines = numericBaselines; }

    public List<Double> baselineFor(String endpoint, String field) {
        return numericBaselines.getOrDefault(endpoint + ":" + field, List.of(-1.0e9, 1.0e9));
    }
}

```

gateway/src/main/java/com/aiosecfirewall/gateway/config/GatewayConfig.java

```

package com.aiosecfirewall.gateway.config;

import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.client.WebClient;

@Configuration
@EnableConfigurationProperties(SecurityProperties.class)
public class GatewayConfig {

    /**
     * Dedicated WebClient for the Threat Engine call. Kept separate from any
     * other outbound client so its connection pool and timeouts never get
     * shared with (or starved by) unrelated traffic.
     */
    @Bean
    public WebClient threatEngineWebClient(
        org.springframework.core.env.Environment env) {
        String baseUrl = env.getProperty("firewall.threat-engine-url", "http://threat-engine:8000");
        return WebClient.builder().baseUrl(baseUrl).build();
    }
}

```

7.4 DTOs: Keeping the Java <-> Python Contract Honest

The Python service is snake_case on the wire (Pydantic's default). Java stays idiomatic camelCase internally and maps explicitly with

`@JsonProperty` so neither service has to bend its own language's conventions to match the other's.

gateway/src/main/java/com/aiosecfirewall/gateway/dto/RequestSignalDTO.java

```

package com.aiosecfirewall.gateway.dto;

import com.fasterxml.jackson.annotation.JsonProperty;
import java.util.List;
import java.util.Map;

/**
 * Mirrors app.features.RequestSignal (Pydantic model) on the Threat Engine
 * side. The Python service is snake_case on the wire (Pydantic default);
 * Java stays camelCase internally and maps explicitly via @JsonProperty so
 * neither side has to bend its own language conventions.
 */
public class RequestSignalDTO {

    @JsonProperty("req_rate_10s")
    public double reqRate10s;

    @JsonProperty("req_rate_60s")
    public double reqRate60s;

    @JsonProperty("distinct_endpoints_60s")
    public int distinctEndpoints60s;

    @JsonProperty("failed_auth_streak")
    public int failedAuthStreak;

    @JsonProperty("session_age_seconds")

```

```

    public double sessionAgeSeconds;

    @JsonProperty("geo_velocity_kmph")
    public double geoVelocityKmph;

    @JsonProperty("known_bad_ua_flag")
    public int knownBadUaFlag;

    @JsonProperty("response_error_rate_5m")
    public double responseErrorRate5m;

    @JsonProperty("off_hours_flag")
    public int offHoursFlag;

    @JsonProperty("raw_body")
    public String rawBody = "";

    @JsonProperty("param_count")
    public int paramCount;

    @JsonProperty("baseline_param_count")
    public int baselineParamCount = 8;

    @JsonProperty("numeric_fields")
    public Map<String, Double> numericFields = Map.of();

    @JsonProperty("baseline_numeric_ranges")
    public Map<String, List<Double>> baselineNumericRanges = Map.of();
}

```

gateway/src/main/java/com/aifsecfirewall/gateway/dto/ScoreResponseDTO.java

```

package com.aifsecfirewall.gateway.dto;

import com.fasterxml.jackson.annotation.JsonProperty;

public class ScoreResponseDTO {

    @JsonProperty("isolation_score")
    public double isolationScore;

    @JsonProperty("xgboost_score")
    public double xgboostScore;

    @JsonProperty("final_score")
    public double finalScore;

    public String decision; // ALLOW | CHALLENGE | BLOCK

    public String reason;
}

```

7.5 JWT Validation

Zero trust means every request proves who it is on every hop — there's no implicit trust just because a request reached the gateway's network. Any parsing, signature, or expiry failure collapses to a clean "not authenticated" rather than leaking an exception past the filter.

gateway/src/main/java/com/aifsecfirewall/gateway/filter/JwtAuthValidator.java

```

package com.aifsecfirewall.gateway.filter;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;

```

```

import java.util.Optional;

/**
 * Zero-trust means every request proves who it is on every hop -- there is
 * no implicit trust just because a request reached the gateway network.
 * This validator is intentionally strict: any parsing/signature/expiry
 * failure returns empty rather than throwing past the filter, so callers
 * always get a clean "authenticated or not" answer.
 */
@Component
public class JwtAuthValidator {

    private final SecretKey signingKey;

    public JwtAuthValidator(@Value("${firewall.jwt-secret}") String secret) {
        this.signingKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    public Optional<Claims> validate(String bearerToken) {
        if (bearerToken == null || !bearerToken.startsWith("Bearer ")) {
            return Optional.empty();
        }
        String token = bearerToken.substring(7);
        try {
            Claims claims = Jwts.parser()
                .verifyWith(signingKey)
                .build()
                .parseSignedClaims(token)
                .getPayload();
            return Optional.of(claims);
        } catch (JwtException | IllegalArgumentException e) {
            return Optional.empty();
        }
    }
}

```

Unit test:

```
gateway/src/test/java/com/aiecfirewall/gateway/filter/JwtAuthValidatorTest.java
```

```

package com.aiecfirewall.gateway.filter;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.junit.jupiter.api.Test;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class JwtAuthValidatorTest {

    private static final String SECRET = "test-secret-key-at-least-32-bytes-long!!";

    @Test
    void validTokenIsAccepted() {
        JwtAuthValidator validator = new JwtAuthValidator(SECRET);
        SecretKey key = Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));
        String token = Jwts.builder()
            .subject("demo_user")
            .issuedAt(new Date())
            .signWith(key)
            .compact();

        Optional<io.jsonwebtoken.Claims> result = validator.validate("Bearer " + token);
        assertTrue(result.isPresent());
        assertEquals("demo_user", result.get().getSubject());
    }

    @Test

```

```

void missingBearerPrefixIsRejected() {
    JwtAuthValidator validator = new JwtAuthValidator(SECRET);
    assertTrue(validator.validate("just-a-raw-token").isEmpty());
}

@Test
void tamperedTokenIsRejected() {
    JwtAuthValidator validator = new JwtAuthValidator(SECRET);
    assertTrue(validator.validate("Bearer not.a.valid.jwt").isEmpty());
}

@Test
void nullTokenIsRejected() {
    JwtAuthValidator validator = new JwtAuthValidator(SECRET);
    assertTrue(validator.validate(null).isEmpty());
}
}

```

7.6 Redis State Service

Every call in this class targets sub-millisecond latency — this is the layer that makes “learn asynchronously, enforce synchronously” actually cheap enough to run on every single request. The rate counter uses a fixed-window `INCR` + `EXPIRE` rather than a true sliding log: it’s a standard, cheap approximation that’s more than good enough for catching bursty traffic, at a fraction of the memory and CPU cost of a precise sliding window.

```
gateway/src/main/java/com/aifirewall/gateway/service/RedisStateService.java
```

```

package com.aifirewall.gateway.service;

import com.aifirewall.gateway.config.SecurityProperties;
import org.springframework.data.redis.core.ReactiveStringRedisTemplate;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Mono;

import java.time.Duration;

/**
 * Everything the gateway needs from Redis on the hot path. Every call here
 * targets sub-millisecond latency -- this is the layer that makes
 * "learn asynchronously (Kafka + ML), enforce synchronously (Redis)"
 * actually cheap enough to run on every single request.
 */
@Service
public class RedisStateService {

    private final ReactiveStringRedisTemplate redis;
    private final SecurityProperties properties;

    public RedisStateService(ReactiveStringRedisTemplate redis, SecurityProperties properties) {
        this.redis = redis;
        this.properties = properties;
    }

    /** 0(1) lookup: is this client IP currently blocklisted? */
    public Mono<Boolean> isBlocked(String clientId) {
        return redis.hasKey("blocklist:" + clientId);
    }

    /** Gateway-side short-circuit: block an offender immediately without waiting on the async pipeline. */
    public Mono<Boolean> blockImmediately(String clientId, String reason) {
        return redis.opsForValue()
            .set("blocklist:" + clientId, reason, Duration.ofSeconds(properties.getBlocklistTtlSeconds()));
    }

    /**
     * Fixed-window counter: INCR + EXPIRE-if-new. Cheap approximation of a
     * sliding window; good enough to catch bursty credential-stuffing
     * traffic without the cost of a true sliding-log implementation.
     * Returns the count AFTER incrementing.
     */
    public Mono<Long> incrementWindowCounter(String clientId, String endpoint, Duration window) {
        String key = "count:" + clientId + ":" + endpoint;
        return redis.opsForValue().increment(key);
    }
}

```

```

        .flatMap(count -> {
            if (count == 1L) {
                return redis.expire(key, window).thenReturn(count);
            }
            return Mono.just(count);
        });
    }

    public Mono<Long> incrementFailedAuthStreak(String identityKey) {
        String key = "failstreak:" + identityKey;
        return redis.opsForValue().increment(key)
            .flatMap(count -> redis.expire(key, Duration.ofMinutes(15)).thenReturn(count));
    }

    public Mono<Boolean> resetFailedAuthStreak(String identityKey) {
        return redis.opsForValue().delete("failstreak:" + identityKey);
    }
}

```

7.7 Kafka Publisher

Fire-and-forget by design: `KafkaTemplate.send()` returns immediately, and a failed publish is logged and dropped rather than retried inline. Losing an occasional audit event is an acceptable trade-off for never adding latency to a real user's request because of a Kafka hiccup.

```
gateway/src/main/java/com/aiosecfirewall/gateway/service/KafkaRequestPublisher.java
```

```

package com.aiosecfirewall.gateway.service;

import com.aiosecfirewall.gateway.dto.RequestSignalDTO;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

import java.time.Instant;
import java.util.Map;
import java.util.UUID;

/**
 * Publishes request telemetry to Kafka for the async scoring / audit path.
 * This NEVER blocks the reactive request chain -- KafkaTemplate.send()
 * returns immediately and we just log failures. Losing an occasional
 * audit event is an acceptable trade-off for never adding latency to a
 * real user's request because of a Kafka hiccup.
 */
@Service
public class KafkaRequestPublisher {

    private static final Logger log = LoggerFactory.getLogger(KafkaRequestPublisher.class);
    private static final String RAW_TOPIC = "api.requests.raw";

    private final KafkaTemplate<String, String> kafkaTemplate;
    private final ObjectMapper objectMapper;

    public KafkaRequestPublisher(KafkaTemplate<String, String> kafkaTemplate, ObjectMapper objectMapper) {
        this.kafkaTemplate = kafkaTemplate;
        this.objectMapper = objectMapper;
    }

    public void publishAsync(String clientId, String method, String endpoint, RequestSignalDTO signal) {
        try {
            String requestId = UUID.randomUUID().toString();
            Map<String, Object> envelope = Map.of(
                "request_id", requestId,
                "client_ip", clientId,
                "method", method,
                "endpoint", endpoint,
                "timestamp", Instant.now().toString(),
                "signal", signal
            );

```

```

        String json = objectMapper.writeValueAsString(envelope);
        kafkaTemplate.send(RAW_TOPIC, clientId, json)
            .whenComplete((result, ex) -> {
                if (ex != null) {
                    log.warn("Failed to publish request telemetry to Kafka: {}", ex.toString());
                }
            });
    } catch (Exception e) {
        log.warn("Failed to serialize request telemetry, dropping: {}", e.toString());
    }
}
}

```

7.8 Threat Engine Client (Resilience4j)

Two independent safety nets protect the request hot path: a hard timeout (the gateway will not hold a user's request hostage waiting on a model call) and a circuit breaker (if the Threat Engine is unhealthy across many requests, stop calling it for a cool-down window instead of paying the timeout cost on every request). On any failure, this fails open — see Section 3.4 for why that's the right default here and where a real system would flip it per-route.

gateway/src/main/java/com/aiosecfirewall/gateway/client/ThreatEngineClient.java

```

package com.aiosecfirewall.gateway.client;

import com.aiosecfirewall.gateway.config.SecurityProperties;
import com.aiosecfirewall.gateway.dto.RequestSignalDTO;
import com.aiosecfirewall.gateway.dto.ScoreResponseDTO;
import io.github.resilience4j.circuitbreaker.CircuitBreaker;
import io.github.resilience4j.circuitbreaker.CircuitBreakerRegistry;
import io.github.resilience4j.reactor.circuitbreaker.operator.CircuitBreakerOperator;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.core.publisher.Mono;

import java.time.Duration;

/**
 * Calls the FastAPI Threat Engine's synchronous /score endpoint.
 *
 * Two independent safety nets protect the request hot-path:
 * 1. A hard timeout (firewall.threat-engine-timeout-ms, default 25ms) --
 *    the gateway will NOT hold a user's request hostage waiting on a
 *    model call.
 * 2. A circuit breaker -- if the Threat Engine is unhealthy or slow
 *    across many requests, stop calling it altogether for a cool-down
 *    window instead of paying the timeout cost on every request.
 *
 * On ANY failure (timeout, circuit open, 5xx) we FAIL OPEN: allow the
 * request through, but mark it as unscored so it is still audited. This
 * is a deliberate availability-over-precision trade-off appropriate for
 * a gateway sitting in front of production traffic -- see the "Design
 * Decisions" section of the write-up for the reasoning and the
 * alternative (fail-closed) for endpoints where that trade-off should
 * flip, e.g. wire transfers.
 */
@Component
public class ThreatEngineClient {

    private static final Logger log = LoggerFactory.getLogger(ThreatEngineClient.class);
    private static final ScoreResponseDTO UNSCORED = fallback();

    private final WebClient webClient;
    private final CircuitBreaker circuitBreaker;
    private final SecurityProperties properties;

    public ThreatEngineClient(WebClient threatEngineWebClient,
                             CircuitBreakerRegistry registry,
                             SecurityProperties properties) {
        this.webClient = threatEngineWebClient;
        this.circuitBreaker = registry.circuitBreaker("threatEngine");
        this.properties = properties;
    }

```

```

    }

    public Mono<ScoreResponseDTO> score(RequestSignalDTO signal) {
        return webClient.post()
            .uri("/score")
            .bodyValue(signal)
            .retrieve()
            .bodyToMono(ScoreResponseDTO.class)
            .timeout(Duration.ofMillis(properties.getThreatEngineTimeoutMs()))
            .transformDeferred(CircuitBreakerOperator.of(circuitBreaker))
            .onErrorResume(ex -> {
                log.warn("Threat Engine unavailable, failing open: {}", ex.toString());
                return Mono.just(UNSCORED);
            });
    }

    private static ScoreResponseDTO fallback() {
        ScoreResponseDTO dto = new ScoreResponseDTO();
        dto.isolationScore = -1;
        dto.xgboostScore = -1;
        dto.finalScore = -1;
        dto.decision = "UNSCORED";
        dto.reason = "Threat engine timeout/circuit-open - allowed with async follow-up";
        return dto;
    }
}

```

7.9 The Core Zero-Trust Filter

This is the centerpiece of the gateway: a single `GlobalFilter` that runs at `Ordered.HIGHEST_PRECEDENCE`, before routing and before any per-route filter, so every request — regardless of which downstream microservice it's ultimately headed for — passes through the exact same pipeline. That “one chokepoint, uniformly enforced” property is what makes this zero-trust rather than a pile of inconsistent per-service auth checks.

Walking through what it does, in order:

1. **Authenticate** (JWT) unless the path is explicitly public (login, register, health check).
2. **Fast synchronous Redis blocklist check** — reject known-bad identities in well under a millisecond, no ML involved.
3. **Buffer and re-wrap the request body** for POST/PUT/PATCH requests using `DataBufferUtils.join` + a `ServerHttpRequestDecorator` — this is the trickiest part of the whole gateway and is exactly the kind of “read the body once, still forward it intact” problem that comes up in real reactive-gateway work.
4. **Cheap sliding-window rate counters** (also Redis), run concurrently via `Mono.zip` rather than sequentially, to keep the added latency as small as possible.
5. **If the endpoint is high-risk OR the rate counter tripped a threshold**, make the bounded synchronous call to the Threat Engine.
6. **Fire-and-forget publish** the full telemetry (including the raw body, for the async path's deeper payload analysis) to Kafka.
7. **Forward** the (possibly re-wrapped) request downstream, or reject with a clear status code and reason.

Note the **feature-parity trade-off** between the sync and async paths: the synchronous call only populates the features the gateway already has cheaply available in Redis and JWT claims (rate, fail-streak, session age, known-bad User-Agent, off-hours) plus whatever numeric-field analysis it can do on the buffered body within budget. `geo_velocity_kmph` is left at a neutral default here — wiring up a real GeoIP lookup (e.g. MaxMind) adds an I/O call that would need its own timeout budget, which is exactly the kind of scope a real implementation would size up separately. `distinct_endpoints_60s` is similarly stubbed to `1` on the sync path (a Redis `SADD`-based version would be a natural follow-up, see Section 16). This means the synchronous call reliably catches classic credential-stuffing signatures (which live entirely in rate/streak/UA/geo features) while relying on the async path a few hundred milliseconds later to catch payload-driven business-logic abuse and add the offender to the blocklist for subsequent requests. Being able to explain why a system's two paths deliberately see different feature sets — instead of treating that as an oversight — is a good interview signal.

```
gateway/src/main/java/com/aisecfirewall/gateway/filter/ZeroTrustFilter.java
```

```

package com.aisecfirewall.gateway.filter;

import com.aisecfirewall.gateway.client.ThreatEngineClient;
import com.aisecfirewall.gateway.config.SecurityProperties;
import com.aisecfirewall.gateway.dto.RequestSignalDTO;
import com.aisecfirewall.gateway.dto.ScoreResponseDTO;
import com.aisecfirewall.gateway.service.KafkaRequestPublisher;
import com.aisecfirewall.gateway.service.RedisStateService;
import com.fasterxml.jackson.databind.ObjectMapper;
import io.jsonwebtoken.Claims;
import org.slf4j.Logger;

```

```

import org.slf4j.LoggerFactory;
import org.springframework.core.io.buffer.DataBuffer;
import org.springframework.core.io.buffer.DataBufferUtils;
import org.springframework.core.Ordered;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.http.HttpStatus;
import org.springframework.http.server.reactive.ServerHttpRequest;
import org.springframework.http.server.reactive.ServerHttpRequestDecorator;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.time.Instant;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;

/**
 * Central ingress inspection point. Every request that reaches this
 * gateway passes through the same pipeline, regardless of which
 * downstream microservice it is ultimately headed for -- that
 * "one chokepoint, uniformly enforced" property is what makes this a
 * zero-trust design rather than a pile of per-service auth checks.
 *
 * Pipeline (see the write-up's sequence diagram for the full picture):
 * 1. Authenticate (JWT) unless the path is explicitly public.
 * 2. Fast synchronous Redis blocklist check -- reject known-bad
 *    identities in well under a millisecond, no ML involved.
 * 3. Cheap sliding-window rate counters (also Redis).
 * 4. If the endpoint is high-risk OR the rate counter tripped a
 *    threshold, make a BOUNDED synchronous call to the Threat Engine.
 * 5. Fire-and-forget publish full telemetry (incl. request body for
 *    POST/PUT/PATCH) to Kafka for the asynchronous, higher-fidelity
 *    scoring pass and the audit trail.
 * 6. Forward the (possibly re-wrapped) request downstream.
 */
@Component
public class ZeroTrustFilter implements GlobalFilter, Ordered {

    private static final Logger log = LoggerFactory.getLogger(ZeroTrustFilter.class);

    private static final Set<String> PUBLIC_PATHS = Set.of(
        "/api/auth/login", "/api/auth/register", "/actuator/health"
    );
    private static final Set<String> KNOWN_AUTOMATION_UA_MARKERS = Set.of(
        "python-requests", "curl/", "scrapy", "headlesschrome", "go-http-client"
    );

    private final JwtAuthValidator jwtAuthValidator;
    private final RedisStateService redisState;
    private final ThreatEngineClient threatEngineClient;
    private final KafkaRequestPublisher kafkaPublisher;
    private final SecurityProperties properties;
    private final ObjectMapper objectMapper;

    public ZeroTrustFilter(JwtAuthValidator jwtAuthValidator,
                          RedisStateService redisState,
                          ThreatEngineClient threatEngineClient,
                          KafkaRequestPublisher kafkaPublisher,
                          SecurityProperties properties,
                          ObjectMapper objectMapper) {
        this.jwtAuthValidator = jwtAuthValidator;
        this.redisState = redisState;
        this.threatEngineClient = threatEngineClient;
        this.kafkaPublisher = kafkaPublisher;
        this.properties = properties;
        this.objectMapper = objectMapper;
    }

    @Override
    public int getOrder() {
        return Ordered.HIGHEST_PRECEDENCE; // runs before routing / any per-route filter
    }

```

```

    }

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        String path = request.getPath().value();
        String method = request.getMethod().name();
        String clientId = resolveClientId(exchange);

        Optional<Claims> claims = PUBLIC_PATHS.contains(path)
            ? Optional.empty()
            : jwtAuthValidator.validate(request.getHeaders().getFirst("Authorization"));

        if (!PUBLIC_PATHS.contains(path) && claims.isEmpty()) {
            return reject(exchange, HttpStatus.UNAUTHORIZED, "Missing or invalid credentials");
        }

        return redisState.isBlocked(clientId).flatMap(blocked -> {
            if (Boolean.TRUE.equals(blocked)) {
                return reject(exchange, HttpStatus.FORBIDDEN, "Client is temporarily blocked");
            }
            return processBody(exchange, chain, clientId, method, path, claims);
        });
    }

    private Mono<Void> processBody(ServerWebExchange exchange, GatewayFilterChain chain,
        String clientId, String method, String path,
        Optional<Claims> claims) {
        boolean mayHaveBody = Set.of("POST", "PUT", "PATCH").contains(method);
        if (!mayHaveBody) {
            return continuePipeline(exchange, chain, clientId, method, path, claims, "");
        }

        return DataBufferUtils.join(exchange.getRequest().getBody())
            .defaultIfEmpty(exchange.getResponse().bufferFactory().wrap(new byte[0]))
            .flatMap(dataBuffer -> {
                byte[] bytes = new byte[dataBuffer.readableByteCount()];
                dataBuffer.read(bytes);
                DataBufferUtils.release(dataBuffer);
                String bodyString = new String(bytes, StandardCharsets.UTF_8);

                // Re-wrap the request so the buffered body can still be
                // streamed downstream exactly once we've read it here.
                ServerHttpRequest decorated = new ServerHttpRequestDecorator(exchange.getRequest()) {
                    @Override
                    public Flux<DataBuffer> getBody() {
                        return Flux.defer(() -> Flux.just(exchange.getResponse().bufferFactory().wrap(bytes)));
                    }
                };
                ServerWebExchange mutated = exchange.mutate().request(decorated).build();
                return continuePipeline(mutated, chain, clientId, method, path, claims, bodyString);
            });
    }

    private Mono<Void> continuePipeline(ServerWebExchange exchange, GatewayFilterChain chain,
        String clientId, String method, String path,
        Optional<Claims> claims, String bodyString) {

        Mono<Long> rate10s = redisState.incrementWindowCounter(clientId, path, Duration.ofSeconds(10));
        Mono<Long> rate60s = redisState.incrementWindowCounter(clientId, path, Duration.ofSeconds(60));
        String identityKey = claims.map(Claims::getSubject).orElse(clientId);
        Mono<Long> failStreak = redisState.incrementFailedAuthStreak(identityKey).defaultIfEmpty(0L);

        return Mono.zip(rate10s, rate60s, failStreak).flatMap(tuple -> {
            RequestSignalDTO signal = buildSignal(
                tuple.getT1(), tuple.getT2(), tuple.getT3(),
                exchange, claims, path, bodyString
            );

            kafkaPublisher.publishAsync(clientId, method, path, signal);

            boolean highRisk = properties.getHighRiskEndpoints().stream().anyMatch(path::startsWith)
                || tuple.getT1() > properties.getRateEscalationThreshold();

            if (!highRisk) {
                return chain.filter(exchange);
            }
        });
    }

```

```

        return threatEngineClient.score(signal).flatMap(result -> {
            if ("BLOCK".equals(result.decision)) {
                // This offender likely stays blocked on their NEXT request too,
                // once the async consumer's verdict lands in Redis -- but we
                // don't wait for that; block THIS request immediately as well.
                return redisState.blockImmediately(clientIp, result.reason)
                    .then(reject(exchange, HttpStatus.FORBIDDEN, "Request blocked by threat engine: " + result.reason));
            }

            ServerWebExchange tagged = tagRisk(exchange, result);
            return chain.filter(tagged);
        });
    });
}

private RequestSignalDTO buildSignal(long rate10s, long rate60s, long failStreak,
    ServerWebExchange exchange, Optional<Claims> claims,
    String path, String bodyString) {
    RequestSignalDTO signal = new RequestSignalDTO();
    signal.reqRate10s = rate10s;
    signal.reqRate60s = rate60s;
    signal.failedAuthStreak = (int) failStreak;
    signal.distinctEndpoints60s = 1; // refined by the async consumer, which sees the full stream
    signal.sessionAgeSeconds = claims.map(c -> {
        Instant issuedAt = c.getIssuedAt() != null ? c.getIssuedAt().toInstant() : Instant.now();
        return (double) Duration.between(issuedAt, Instant.now()).getSeconds();
    }).orElse(0.0);
    signal.geoVelocityKmph = 0.0; // wire up a GeoIP lookup (e.g. MaxMind) here in production
    signal.knownBadUaFlag = isKnownAutomationUa(exchange) ? 1 : 0;
    signal.responseErrorRate5m = 0.0; // populated by the response-side hook, see RESOURCES in the write-up
    signal.offHoursFlag = isOffHours() ? 1 : 0;
    signal.rawBody = bodyString;
    signal.paramCount = countTopLevelFields(bodyString);
    signal.baselineParamCount = 8;

    Map<String, Double> numericFields = extractNumericFields(bodyString);
    signal.numericFields = numericFields;
    signal.baselineNumericRanges = numericFields.keySet().stream()
        .collect(java.util.stream.Collectors.toMap(
            f -> f,
            f -> properties.baselineFor(path, f)
        ));
    return signal;
}

private ServerWebExchange tagRisk(ServerWebExchange exchange, ScoreResponseDTO result) {
    exchange.getResponse().getHeaders().add("X-Risk-Level", result.decision);
    exchange.getResponse().getHeaders().add("X-Risk-Score", String.valueOf(result.finalScore));
    return exchange;
}

private int countTopLevelFields(String body) {
    if (body == null || body.isBlank()) return 0;
    try {
        Map<?, ?> map = objectMapper.readValue(body, Map.class);
        return map.size();
    } catch (Exception e) {
        return 0;
    }
}

private Map<String, Double> extractNumericFields(String body) {
    if (body == null || body.isBlank()) return Map.of();
    try {
        Map<?, ?> map = objectMapper.readValue(body, Map.class);
        return map.entrySet().stream()
            .filter(e -> e.getValue() instanceof Number)
            .collect(java.util.stream.Collectors.toMap(
                e -> String.valueOf(e.getKey()),
                e -> ((Number) e.getValue()).doubleValue()
            ));
    } catch (Exception e) {
        return Map.of();
    }
}
}

```

```

private boolean isKnownAutomationUa(ServerWebExchange exchange) {
    String ua = exchange.getRequest().getHeaders().getFirst("User-Agent");
    if (ua == null) return true; // no UA at all is itself a signal
    String lower = ua.toLowerCase();
    return KNOWN_AUTOMATION_UA_MARKERS.stream().anyMatch(lower::contains);
}

private boolean isOffHours() {
    int hour = java.time.LocalTime.now(java.time.ZoneOffset.UTC).getHour();
    return hour < 6 || hour > 22;
}

private String resolveClientIp(ServerWebExchange exchange) {
    String forwarded = exchange.getRequest().getHeaders().getFirst("X-Forwarded-For");
    if (forwarded != null && !forwarded.isBlank()) {
        return forwarded.split(",")[0].trim();
    }
    return Optional.ofNullable(exchange.getRequest().getRemoteAddress())
        .map(addr -> addr.getAddress().getHostAddress())
        .orElse("unknown");
}

private Mono<Void> reject(ServerWebExchange exchange, HttpStatus status, String reason) {
    exchange.getResponse().setStatusCode(status);
    exchange.getResponse().getHeaders().add("Content-Type", "application/json");
    byte[] body = ("{"error\":\"" + reason + "\"}").getBytes(StandardCharsets.UTF_8);
    DataBuffer buffer = exchange.getResponse().bufferFactory().wrap(body);
    return exchange.getResponse().writeWith(Mono.just(buffer));
}
}

```

7.10 Application Configuration

gateway/src/main/resources/application.yml

```

server:
  port: 8080

spring:
  application:
    name: ai-sec-gateway

cloud:
  gateway:
    routes:
      - id: auth-service
        uri: http://auth-service:8080
        predicates:
          - Path=/api/auth/**
      - id: order-service
        uri: http://order-service:8080
        predicates:
          - Path=/api/orders/**
      - id: checkout-service
        uri: http://order-service:8080
        predicates:
          - Path=/api/checkout/**

data:
  redis:
    host: redis
    port: 6379
    timeout: 2s

kafka:
  bootstrap-servers: kafka:9092
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.apache.kafka.common.serialization.StringSerializer
    acks: 1
    retries: 3

management:

```

```

endpoints:
  web:
    exposure:
      include: health, info, prometheus, metrics
    endpoint:
      health:
        show-details: always

resilience4j:
  circuitbreaker:
    instances:
      threatEngine:
        sliding-window-size: 50
        minimum-number-of-calls: 20
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 5

firewall:
  jwt-secret: "CHANGE_ME_TO_A_LONG_RANDOM_SECRET_LOADED_FROM_A_VAULT"
  threat-engine-url: "http://threat-engine:8000"
  threat-engine-timeout-ms: 25
  rate-escalation-threshold: 15
  blocklist-ttl-seconds: 900
  high-risk-endpoints:
    - /api/auth/login
    - /api/checkout
  numeric-baselines:
    "/api/checkout:qty": [1, 10]
    "/api/checkout:price": [1, 10000]
    "/api/checkout:discount_pct": [0, 30]

logging:
  level:
    com.aisecfirewall.gateway: INFO

```

7.11 Dockerfile

A standard multi-stage build: compile with Maven in a throwaway build image, then copy only the final JAR into a slim JRE runtime image. This keeps the shipped image small and avoids bundling the entire Maven toolchain into what actually runs in production.

gateway/Dockerfile

```

# ---- Build stage ----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build
COPY pom.xml .
# Cache dependencies separately from source so code edits don't re-download the internet
RUN mvn -B dependency:go-offline
COPY src ./src
RUN mvn -B clean package -DskipTests

# ---- Runtime stage ----
FROM eclipse-temurin:17-jre-jammy
WORKDIR /app
RUN addgroup --system spring && adduser --system --ingroup spring spring
USER spring:spring
COPY --from=build /build/target/gateway-*.jar app.jar
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=3s --start-period=30s \
  CMD wget -qO- http://localhost:8080/actuator/health | grep -q '"status":"UP"' || exit 1
ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]

```

8. Phase 3 — Mock Backend Microservices

These stand in for the “internal microservices” the gateway is protecting. They are deliberately trivial — their only job is to exist behind the gateway so the attack simulations in Section 14 can prove a negative: a blocked request should show up in Elasticsearch as blocked and should **never** reach these services’ logs at all. Point the gateway’s routes at your own real services instead once you’re past the demo stage.

8.1 Mock Auth Service

Issues a JWT for a single fixed demo credential — deliberately, so the credential-stuffing simulation has something meaningful to brute-force against.

mock-services/auth-service/main.py

```
"""
Minimal stand-in for a real identity/auth microservice. Its only job here
is to issue a JWT the gateway can validate, and to deliberately accept a
FIXED demo credential so the attack-simulation scripts have something to
brute-force -- that's what makes the credential-stuffing demo meaningful.

Do not use this as-is anywhere near production: no password hashing, no
lockout, no rate limiting of its own (that is precisely the gateway's
job, which is the point of this whole project).
"""

import os
import time

import jwt
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="mock-auth-service")

JWT_SECRET = os.getenv("JWT_SECRET", "CHANGE_ME_TO_A_LONG_RANDOM_SECRET_LOADED_FROM_A_VAULT")
DEMO_USERNAME = "demo_user"
DEMO_PASSWORD = "correct-horse-battery-staple"

class LoginRequest(BaseModel):
    username: str
    password: str

@app.post("/api/auth/login")
def login(req: LoginRequest):
    if req.username == DEMO_USERNAME and req.password == DEMO_PASSWORD:
        token = jwt.encode({"sub": req.username, "iat": int(time.time())}, JWT_SECRET, algorithm="HS256")
        return {"access_token": token, "token_type": "bearer"}
    raise HTTPException(status_code=401, detail="Invalid credentials")

@app.post("/api/auth/register")
def register():
    return {"status": "registration disabled in demo"}

@app.get("/health")
def health():
    return {"status": "ok"}
```

mock-services/auth-service/requirements.txt

```
fastapi==0.110.0
uvicorn[standard]==0.29.0
pyjwt==2.8.0
```

mock-services/auth-service/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py .
EXPOSE 8080
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

8.2 Mock Order / Checkout Service

mock-services/order-service/main.py

```
"""
Minimal stand-in for a real orders/checkout microservice. It does nothing
clever -- it just echoes back what it received plus a timestamp. Its only
purpose in this project is to be the thing sitting BEHIND the gateway, so
the attack-simulation scripts can prove a negative: a blocked request
should show up in Elasticsearch as blocked and should NEVER show up in
this service's logs at all.
"""

import time
from fastapi import FastAPI, Request

app = FastAPI(title="mock-order-service")

@app.post("/api/orders")
async def create_order(request: Request):
    body = await request.json()
    print(f"[order-service] received order: {body}")
    return {"status": "accepted", "received": body, "server_time": time.time()}

@app.post("/api/checkout")
async def checkout(request: Request):
    body = await request.json()
    print(f"[order-service] received checkout: {body}")
    return {"status": "accepted", "received": body, "server_time": time.time()}

@app.get("/health")
def health():
    return {"status": "ok"}
```

mock-services/order-service/requirements.txt

```
fastapi==0.110.0
uvicorn[standard]==0.29.0
```

mock-services/order-service/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py .
EXPOSE 8080
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8080"]
```

9. Phase 4 — Audit Logging (Elasticsearch)

9.1 Index Template and ILM Policy

The index template defines the mapping so `client_ip` is queryable as an actual IP type (enabling CIDR-range queries), scores are floats, and `decision` / `endpoint` / `method` are keywords (exact-match, aggregatable — what you want for a Kibana “top blocked IPs” panel). The ILM policy handles retention automatically: roll over daily or at 5GB, shrink after 3 days, delete after 30 — so the index doesn’t grow unbounded on a system nobody is pruning by hand.

elasticsearch/audit-events-template.json

```
{
  "index_patterns": ["api-audit-events-*"],
  "template": {
    "settings": {
      "number_of_shards": 1,
      "number_of_replicas": 0,
      "index.lifecycle.name": "api-audit-ilm-policy"
    },
    "mappings": {
      "properties": {
        "request_id": { "type": "keyword" },
        "client_ip": { "type": "ip" },
        "endpoint": { "type": "keyword" },
        "method": { "type": "keyword" },
        "timestamp": { "type": "date" },
        "isolation_score": { "type": "float" },
        "xgboost_score": { "type": "float" },
        "final_score": { "type": "float" },
        "decision": { "type": "keyword" },
        "reason": { "type": "text" }
      }
    }
  }
}
```

elasticsearch/ilm-policy.json

```
{
  "policy": {
    "phases": {
      "hot": {
        "min_age": "0ms",
        "actions": {
          "rollover": { "max_age": "1d", "max_primary_shard_size": "5gb" }
        }
      },
      "warm": {
        "min_age": "3d",
        "actions": { "shrink": { "number_of_shards": 1 } }
      },
      "delete": {
        "min_age": "30d",
        "actions": { "delete": {} }
      }
    }
  }
}
```

9.2 Audit Indexer

A small, single-responsibility service: read scored events off `api.audit.events` and bulk-index them into a daily-rolling `api-audit-events-YYYY.MM.DD` index. It's kept deliberately separate from the Threat Engine so the two can scale, fail, and deploy independently — a burst of indexing load (e.g. a Kibana backfill) should never be able to slow down real-time scoring, and vice versa. At real scale you'd likely replace this hand-rolled consumer with a managed Kafka Connect Elasticsearch sink connector or Logstash; it's built by hand here because it's small enough to fully understand and makes the data flow explicit for a portfolio walkthrough.

audit-indexer/indexer.py

```
"""
indexer.py
Single-responsibility service: read scored events off `api.audit.events`
and bulk-index them into Elasticsearch under a daily-rolling index
(api-audit-events-YYYY.MM.DD).

Kept deliberately separate from the Threat Engine so the two services can
scale, fail, and deploy independently -- a burst of audit-indexing load
(e.g. Kibana backfill) should never be able to slow down real-time
scoring, and vice versa. In a larger deployment this is exactly the kind
of thing you would eventually replace with Logstash, Filebeat, or a
managed Kafka Connect Elasticsearch sink connector; a hand-rolled
consumer is shown here because it's small enough to fully understand and
makes the data flow explicit for a portfolio/interview walkthrough.
"""

from __future__ import annotations
import asyncio
import json
import logging
import os
from datetime import datetime, timezone

from aiokafka import AIOKafkaConsumer
from elasticsearch import AsyncElasticsearch
from elasticsearch.helpers import async_bulk

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s: %(message)s")
logger = logging.getLogger("audit-indexer")

KAFKA_BROKERS = os.getenv("KAFKA_BROKERS", "kafka:9092")
ES_URL = os.getenv("ELASTICSEARCH_URL", "http://elasticsearch:9200")
AUDIT_TOPIC = os.getenv("AUDIT_TOPIC", "api.audit.events")
FLUSH_INTERVAL_SECONDS = float(os.getenv("FLUSH_INTERVAL_SECONDS", "2.0"))
FLUSH_BATCH_SIZE = int(os.getenv("FLUSH_BATCH_SIZE", "200"))

def _index_name() -> str:
    return f"api-audit-events-{datetime.now(timezone.utc):%Y.%m.%d}"

async def _flush(es: AsyncElasticsearch, buffer: list[dict]) -> None:
    if not buffer:
        return
    actions = [{"_index": _index_name(), "_source": event} for event in buffer]
    try:
        successes, errors = await async_bulk(es, actions, raise_on_error=False)
        if errors:
            logger.warning("Indexed %d docs, %d failed", successes, len(errors))
        else:
            logger.info("Indexed %d docs into %s", successes, _index_name())
    except Exception:
        logger.exception("Bulk index call failed; %d events dropped from this batch", len(buffer))
    finally:
        buffer.clear()

async def run() -> None:
    es = AsyncElasticsearch(hosts=[ES_URL])
    consumer = AIOKafkaConsumer(
        AUDIT_TOPIC,
        bootstrap_servers=KAFKA_BROKERS,
        group_id="audit-indexer",
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
    )
    consumer.start()
    while True:
        messages = consumer.poll(timeout=1.0)
        for message in messages.values():
            buffer.append(message.value)
            if len(buffer) == FLUSH_BATCH_SIZE:
                await _flush(es, buffer)
                buffer.clear()
        if not messages:
            await asyncio.sleep(FLUSH_INTERVAL_SECONDS)
```

```

        auto_offset_reset="latest",
        enable_auto_commit=True,
    )

    while True:
        try:
            await consumer.start()
            logger.info("Connected to Kafka, consuming '%s'", AUDIT_TOPIC)
            break
        except Exception as exc:
            logger.warning("Kafka not ready (%s); retrying in 5s", exc)
            await asyncio.sleep(5)

    buffer: list[dict] = []
    try:
        while True:
            try:
                msg = await asyncio.wait_for(consumer.getone(), timeout=FLUSH_INTERVAL_SECONDS)
                buffer.append(msg.value)
                if len(buffer) >= FLUSH_BATCH_SIZE:
                    await _flush(es, buffer)
            except asyncio.TimeoutError:
                await _flush(es, buffer) # time-based flush even if the batch isn't full
    finally:
        await _flush(es, buffer)
        await consumer.stop()
        await es.close()

if __name__ == "__main__":
    asyncio.run(run())

```

```
audit-indexer/requirements.txt
```

```

aiokafka==0.10.0
elasticsearch==8.13.0

```

```
audit-indexer/Dockerfile
```

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY indexer.py .
CMD ["python", "indexer.py"]

```

10. Phase 5 — Full Docker Compose Orchestration

This is what ties every service in the previous phases together. A few details worth calling out: Kafka runs in **KRaft mode** (no separate Zookeeper container needed — this was a real operational simplification introduced in modern Kafka versions and is worth mentioning as current knowledge in an interview). `depends_on` uses `condition: service_healthy` wherever a healthcheck exists, so, for example, the gateway doesn't start attempting Kafka connections before Kafka has actually finished its KRaft bootstrap.

docker-compose.yml

```
name: ai-api-security-firewall

networks:
  firewall-net:
    driver: bridge

volumes:
  es-data:

services:
  # -----
  # INGESTION: single-node Kafka in KRaft mode (no Zookeeper needed)
  # -----
  kafka:
    image: apache/kafka:3.7.0
    container_name: kafka
    networks: [firewall-net]
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      CLUSTER_ID: "MkU30EVbNTcwNTJjENDM2Qk"
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092 || exit 1"]
      interval: 10s
      timeout: 10s
      retries: 10

  # -----
  # STATE TRACKING: live blocklist + sliding-window rate counters
  # -----
  redis:
    image: redis:7-alpine
    container_name: redis
    networks: [firewall-net]
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

  # -----
  # AUDIT LOGGING: analyzed-traffic + blocked-request index
  # -----
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.13.0
    container_name: elasticsearch
    networks: [firewall-net]
    environment:
      discovery.type: single-node
      xpack.security.enabled: "false"
```

```

    ES_JAVA_OPTS: "-Xms512m -Xmx512m"
  ports:
    - "9200:9200"
  volumes:
    - es-data:/usr/share/elasticsearch/data
  healthcheck:
    test: ["CMD-SHELL", "curl -sf http://localhost:9200/_cluster/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 15

kibana:
  image: docker.elastic.co/kibana/kibana:8.13.0
  container_name: kibana
  networks: [firewall-net]
  depends_on:
    elasticsearch:
      condition: service_healthy
  environment:
    ELASTICSEARCH_HOSTS: http://elasticsearch:9200
  ports:
    - "5601:5601"

# -----
# THREAT ENGINE: FastAPI + Isolation Forest / XGBoost ensemble
# -----
threat-engine:
  build: ./threat-engine
  container_name: threat-engine
  networks: [firewall-net]
  depends_on:
    kafka:
      condition: service_healthy
    redis:
      condition: service_healthy
  environment:
    KAFKA_BROKERS: kafka:9092
    REDIS_URL: redis://redis:6379/0
    RAW_TOPIC: api.requests.raw
    AUDIT_TOPIC: api.audit.events
    BLOCKLIST_TTL_SECONDS: "900"
  ports:
    - "8000:8000"
  healthcheck:
    test: ["CMD-SHELL", "python -c \"import urllib.request; urllib.request.urlopen('http://localhost:8000/health')\" || exit 1"]
    interval: 15s
    timeout: 5s
    retries: 10

# -----
# AUDIT INDEXER: Kafka -> Elasticsearch bulk shipper
# -----
audit-indexer:
  build: ./audit-indexer
  container_name: audit-indexer
  networks: [firewall-net]
  depends_on:
    kafka:
      condition: service_healthy
    elasticsearch:
      condition: service_healthy
  environment:
    KAFKA_BROKERS: kafka:9092
    ELASTICSEARCH_URL: http://elasticsearch:9200
    AUDIT_TOPIC: api.audit.events

# -----
# MOCK BACKENDS: stand-ins for real internal microservices
# -----
auth-service:
  build: ./mock-services/auth-service
  container_name: auth-service
  networks: [firewall-net]
  environment:
    JWT_SECRET: "CHANGE_ME_TO_A_LONG_RANDOM_SECRET_LOADED_FROM_A_VAULT"

```

```

order-service:
  build: ./mock-services/order-service
  container_name: order-service
  networks: [firewall-net]

# -----
# ROUTING: Spring Boot zero-trust gateway (the single ingress point)
# -----
gateway:
  build: ./gateway
  container_name: gateway
  networks: [firewall-net]
  depends_on:
    kafka:
      condition: service_healthy
    redis:
      condition: service_healthy
    threat-engine:
      condition: service_healthy
    auth-service:
      condition: service_started
    order-service:
      condition: service_started
  environment:
    SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    SPRING_DATA_REDIS_HOST: redis
    FIREWALL_THREAT-ENGINE-URL: http://threat-engine:8000
    FIREWALL_JWT-SECRET: "CHANGE_ME_TO_A_LONG_RANDOM_SECRET_LOADED_FROM_A_VAULT"
  ports:
    - "8080:8080"
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q '\"status\": \"UP\"' || exit 1"]
    interval: 15s
    timeout: 5s
    retries: 10

```

11. Phase 6 — Infrastructure Bootstrap

Kafka *would* auto-create topics on first publish with default settings, but explicitly creating them with defined partition counts is the right habit — it's the difference between “it happened to work” and “I decided this topic needs 3 partitions for parallelism.” Run this once, a short while after

`docker compose up -d` (give Elasticsearch and Kafka roughly 30 seconds to finish starting first):

scripts/setup_infra.sh

```
#!/usr/bin/env bash
# One-time provisioning: run this once after `docker compose up -d` (give
# Elasticsearch and Kafka ~30s to finish starting first). Safe to re-run --
# every call here is idempotent (PUT-based).
set -euo pipefail

ES_URL="${ES_URL:-http://localhost:9200}"

echo "==> Waiting for Elasticsearch..."
until curl -sf "${ES_URL}/_cluster/health" > /dev/null; do
    sleep 2
done

echo "==> Creating ILM policy (api-audit-ilm-policy)"
curl -sf -X PUT "${ES_URL}/_ilm/policy/api-audit-ilm-policy" \
    -H "Content-Type: application/json" \
    --data-binary @elasticsearch/ilm-policy.json | jq . || true

echo "==> Creating index template (api-audit-events-*)"
curl -sf -X PUT "${ES_URL}/_index_template/api-audit-events-template" \
    -H "Content-Type: application/json" \
    --data-binary @elasticsearch/audit-events-template.json | jq . || true

echo "==> Creating Kafka topics"
docker exec kafka /opt/kafka/bin/kafka-topics.sh --create --if-not-exists \
    --bootstrap-server localhost:9092 --topic api.requests.raw \
    --partitions 3 --replication-factor 1
docker exec kafka /opt/kafka/bin/kafka-topics.sh --create --if-not-exists \
    --bootstrap-server localhost:9092 --topic api.audit.events \
    --partitions 3 --replication-factor 1

echo "==> Done. Topics:"
docker exec kafka /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

12. Phase 7 — Running It End to End

```
git clone <your-repo-url>
cd ai-api-security-firewall

# 1. Build and start every service
docker compose up -d --build

# 2. Watch the gateway and threat-engine come up healthy
docker compose ps

# 3. Provision Kafka topics + the Elasticsearch index template/ILM policy
chmod +x scripts/setup_infra.sh
./scripts/setup_infra.sh

# 4. Get a JWT for the demo user
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"demo_user","password":"correct-horse-battery-staple"}'

# 5. Use the token to place a normal order (should succeed, HTTP 200)
curl -X POST http://localhost:8080/api/orders \
  -H "Authorization: Bearer <token from step 4>" \
  -H "Content-Type: application/json" \
  -d '{"item_id": 101, "qty": 1, "price": 29.99}'
```

If step 5 returns a 200 with the order echoed back, every service in the pipeline — gateway, Redis, Kafka, Threat Engine, and the mock order service — is wired correctly. Now break it on purpose (Section 14).

13. Endpoint Reference

Method	Path	Service	Auth required	Notes
POST	/api/auth/login	gateway -> auth-service	No	Public path
POST	/api/auth/register	gateway -> auth-service	No	Public path (disabled in demo)
POST	/api/orders	gateway -> order-service	Yes	Normal-risk
POST	/api/checkout	gateway -> order-service	Yes	High-risk (always synchronously scored)
POST	/score	Threat Engine (internal)	No (internal network only)	Called by the gateway, not exposed publicly
GET	/health	Threat Engine	No	Liveness/readiness
GET	/actuator/health	Gateway	No	Liveness/readiness
GET	http://localhost:5601	Kibana	No	Local demo only

14. Phase 8 — Attack Simulation & Verification

A security project without a demonstrable attack is just a diagram. These scripts are the executable proof that the pipeline actually works — run them, then check the results.

14.1 Credential Stuffing Simulation

Fires a burst of login attempts using a rotating small wordlist — the classic shape of the attack: high volume, mostly-failing auth, hammering a single endpoint, often carrying an automation-tool User-Agent.

```
scripts/simulate_credential_stuffing.py
```

```
"""
simulate_credential_stuffing.py

Fires a burst of login attempts at the gateway using a rotating list of
usernames and passwords pulled from a small local wordlist -- the classic
shape of a credential-stuffing attack (high volume, mostly-failing auth,
hammering a single endpoint, often from automation tooling).

Usage:
python scripts/simulate_credential_stuffing.py --target http://localhost:8080 --requests 60

Expected result against a running stack:
- The first several requests get a normal 401 (bad credentials) while
  the synchronous score stays under the CHALLENGE threshold.
- Within a few dozen requests, req_rate_10s and failed_auth_streak push
  the combined score past BLOCK, and you should start seeing 403s with
  "Request blocked by threat engine" bodies.
- Check Kibana / the Elasticsearch index directly afterward: you should
  see a cluster of decision=BLOCK events from this script's source IP.
"""

import argparse
import asyncio
import random
import time

import httpx

USERNAMES = ["admin", "demo_user", "root", "test", "administrator", "jsmith", "support"]
PASSWORDS = [
    "123456", "password", "letmein", "qwerty123", "Passw0rd!", "welcome1",
    "correct-horse-battery-staple", # the one real valid demo password, mixed in on purpose
]

async def attempt(client: httpx.AsyncClient, target: str, i: int):
    username = random.choice(USERNAMES)
    password = random.choice(PASSWORDS)
    try:
        resp = await client.post(
            f"{target}/api/auth/login",
            json={"username": username, "password": password},
            headers={"User-Agent": "python-requests/credential-stuffing-demo"},
            timeout=5.0,
        )
        risk = resp.headers.get("X-Risk-Level", "-")
        print(f"[{i:03d}] user={username:14s} -> HTTP {resp.status_code} X-Risk-Level={risk}")
    except httpx.HTTPError as e:
        print(f"[{i:03d}] request failed: {e}")

async def main(target: str, total_requests: int, delay_ms: int):
    async with httpx.AsyncClient() as client:
        for i in range(total_requests):
            await attempt(client, target, i)
            await asyncio.sleep(delay_ms / 1000)

if __name__ == "__main__":
```

```

parser = argparse.ArgumentParser(description="Simulate a credential-stuffing burst against the gateway.")
parser.add_argument("--target", default="http://localhost:8080")
parser.add_argument("--requests", type=int, default=60)
parser.add_argument("--delay-ms", type=int, default=150, help="delay between attempts, in milliseconds")
args = parser.parse_args()

start = time.time()
asyncio.run(main(args.target, args.requests, args.delay_ms))
print(f"\nDone in {time.time() - start:.1f}s")

```

```
python scripts/simulate_credential_stuffing.py --target http://localhost:8080 --requests 60
```

What you should observe: the first several attempts return a normal 401 with the risk header absent or low. Within a few dozen requests, `req_rate_10s` and `failed_auth_streak` push the combined score past the `BLOCK` threshold, and the response shifts to 403 with `X-Risk-Level: BLOCK`. Check `scripts/query_audit_trail.py` afterward — you should see a cluster of `BLOCK` events from the script's source IP.

14.2 Business-Logic Abuse Simulation

Sends a mix of normal and tampered checkout payloads at a low, human-paced rate — the scenario a rate-limiter or signature-based WAF cannot catch, because nothing here looks like a flood and there's no attack-shaped string to pattern-match.

```
scripts/simulate_business_logic_abuse.py
```

```

"""
simulate_business_logic_abuse.py

Sends a mix of NORMAL checkout requests and TAMPERED ones (negative
quantity, near-zero price, extra hidden fields like a fake
"admin_override" flag) at a LOW, human-looking request rate. This is the
scenario a rate-limiter or a signature-based WAF cannot catch: nothing
here looks like a flood, and there is no SQL-injection-shaped string to
pattern-match against. The only tell is that the numeric fields fall
outside the range a legitimate checkout would ever produce, and that the
payload shape (extra params) doesn't match the endpoint's baseline.

Usage:
    python scripts/simulate_business_logic_abuse.py --target http://localhost:8080 --token <JWT>

Get a token first:
    curl -X POST http://localhost:8080/api/auth/login \
        -H "Content-Type: application/json" \
        -d '{"username":"demo_user","password":"correct-horse-battery-staple"}'
"""
import argparse
import random
import time

import httpx

NORMAL_ORDERS = [
    {"item_id": 101, "qty": 1, "price": 29.99},
    {"item_id": 205, "qty": 2, "price": 14.50},
    {"item_id": 310, "qty": 1, "price": 89.00},
]

TAMPERED_ORDERS = [
    {"item_id": 101, "qty": -5, "price": 0.01, "coupon_stack": 12, "admin_override": True},
    {"item_id": 205, "qty": 999, "price": 0.00, "internal_test_mode": True},
    {"item_id": 310, "qty": 1, "price": -89.00, "refund_now": True},
]

def send_batch(target: str, token: str, orders: list[dict], label: str):
    headers = {"Authorization": f"Bearer {token}" if token else {}}
    with httpx.Client() as client:
        for i, order in enumerate(orders):
            resp = client.post(f"{target}/api/checkout", json=order, headers=headers, timeout=5.0)
            risk = resp.headers.get("X-Risk-Level", "-")
            print(f"[{label} {i}] payload={order} -> HTTP {resp.status_code} X-Risk-Level={risk}")

```

```

        time.sleep(random.uniform(0.5, 1.5)) # deliberately slow / human-paced

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Simulate business-logic abuse against the checkout endpoint.")
    parser.add_argument("--target", default="http://localhost:8080")
    parser.add_argument("--token", default="", help="JWT from /api/auth/login")
    args = parser.parse_args()

    print("--- sending normal traffic (should ALLOW) ---")
    send_batch(args.target, args.token, NORMAL_ORDERS, "normal")

    print("\n--- sending tampered traffic (should CHALLENGE/BLOCK) ---")
    send_batch(args.target, args.token, TAMPERED_ORDERS, "tampered")

```

```

TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"demo_user","password":"correct-horse-battery-staple"}' | jq -r .access_token)

python scripts/simulate_business_logic_abuse.py --target http://localhost:8080 --token "$TOKEN"

```

What you should observe: the three normal orders return 200 with a low or absent risk header. The three tampered orders (negative quantity, \$0.00/\$0.01 price, extra hidden fields) should come back tagged `X-Risk-Level: CHALLENGE` or blocked outright with 403, depending on how far outside the configured `numeric-baselines` each payload falls.

14.3 Verifying the Audit Trail

scripts/query_audit_trail.py

```

"""
query_audit_trail.py
Quick sanity-check query against Elasticsearch after running one of the
attack-simulation scripts: prints the most recent BLOCK/CHALLENGE
decisions so you can see the pipeline actually worked end-to-end without
opening Kibana.

Usage:
    python scripts/query_audit_trail.py --es http://localhost:9200
"""
import argparse
import json
import httpx

QUERY = {
    "size": 20,
    "sort": [{"timestamp": "desc"}],
    "query": {
        "bool": {
            "must_not": [{"term": {"decision": "ALLOW"}}]
        }
    },
}

def main(es_url: str):
    resp = httpx.post(
        f"{es_url}/api-audit-events-*/_search",
        json=QUERY,
        timeout=10.0,
    )
    resp.raise_for_status()
    hits = resp.json().get("hits", {}).get("hits", [])
    if not hits:
        print("No BLOCK/CHALLENGE events found yet. Run an attack simulation script first.")
        return

    print(f"{'timestamp':>25s} {'client_ip':>15s} {'decision':>10s} {'final_score':>11s} reason")
    print("-" * 100)
    for hit in hits:
        src = hit["_source"]

```

```

        print(
            f"{{src.get('timestamp',''):25s}} "
            f"{{src.get('client_ip',''):15s}} "
            f"{{src.get('decision',''):10s}} "
            f"{{src.get('final_score', 0):>11.3f}} "
            f"{{src.get('reason','')}}\n"
        )

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--es", default="http://localhost:9200")
    args = parser.parse_args()
    main(args.es)

```

```
python scripts/query_audit_trail.py --es http://localhost:9200
```

Or open Kibana at <http://localhost:5601>, create a data view over `api-audit-events-*`, and build a simple dashboard: a time-series of `final_score`, a terms aggregation on `decision`, and a table of the top 10 `client_ip` values by blocked-request count. This is a five-minute task that turns into a genuinely good portfolio screenshot.

14.4 Load Testing the Latency Budget

The write-up claims a ~25ms synchronous budget and sub-millisecond Redis checks. Don't just assert that — measure it.

```
scripts/locustfile.py
```

```

"""
locustfile.py
Baseline latency/throughput test for the gateway's benign-traffic path.
This is what you run to defend the "sub-millisecond" and "25ms budget"
claims in the write-up with an actual number instead of an assertion.

Usage:
  pip install locust
  locust -f scripts/locustfile.py --host http://localhost:8080 \
    --users 50 --spawn-rate 10 --run-time 1m --headless
"""
from locust import HttpUser, task, between

class GatewayUser(HttpUser):
    wait_time = between(0.2, 1.0)

    @task(3)
    def browse_orders(self):
        self.client.post(
            "/api/orders",
            json={"item_id": 101, "qty": 1, "price": 29.99},
            headers={"Authorization": "Bearer REPLACE_WITH_VALID_TOKEN"},
            name="/api/orders [benign]",
        )

    @task(1)
    def login(self):
        self.client.post(
            "/api/auth/login",
            json={"username": "demo_user", "password": "correct-horse-battery-staple"},
            name="/api/auth/login [benign]",
        )

```

```

pip install locust
locust -f scripts/locustfile.py --host http://localhost:8080 \
  --users 50 --spawn-rate 10 --run-time 1m --headless

```

Report the p50/p95/p99 latencies Locust prints out in your README or resume bullet instead of a made-up number — “p95 latency of 42ms under 50 concurrent users” is a far stronger line than “low latency,” and it’s one you can defend if asked how you measured it.

15. Phase 9 — CI/CD Pipeline

A CI pipeline that actually trains the model and runs both test suites on every push is a meaningfully stronger signal than a README that claims “tested” with no evidence. This one has three jobs: train + test the Python service, build + test the Java service, and finally build every Dockerfile to catch container-level breakage the unit tests wouldn’t.

`.github/workflows/ci.yml`

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  threat-engine:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        working-directory: threat-engine
        run: pip install -r requirements-dev.txt
      - name: "Train model (also produces the artifacts the tests score against)"
        working-directory: threat-engine
        run: python train/train_model.py
      - name: Unit tests
        working-directory: threat-engine
        run: pytest -q

  gateway:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"
          cache: maven
      - name: Build and unit test
        working-directory: gateway
        run: mvn -B clean verify

  docker-build:
    runs-on: ubuntu-latest
    needs: [threat-engine, gateway]
    steps:
      - uses: actions/checkout@v4
      - name: Build all images (no push - validates every Dockerfile)
        run: docker compose build
```

16. Phase 10 — Production Hardening & Roadmap

Everything in this guide runs happily on a laptop. Here's the honest gap list between that and something you'd actually put in front of real traffic — treat this as your project's roadmap, and as material for the "how would you scale this" interview question.

16.1 Sample Kubernetes Manifest

The natural next step after Compose. Included here as a concrete artifact rather than a promise — a Deployment with 3 replicas, a readiness/liveness

probe against `/actuator/health`, resource requests/limits, and an HPA that scales 3-12 pods on CPU utilization.

k8s/gateway-deployment.yaml

```
# Illustrative K8s manifest for the gateway -- the natural next step after
# docker-compose for a real deployment. Not wired into the docker-compose
# demo; included so the write-up's "Production Hardening" section has a
# concrete artifact to point at rather than just prose. Adjust the image
# reference to wherever your CI pipeline pushes it.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-sec-gateway
  labels:
    app: ai-sec-gateway
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ai-sec-gateway
  template:
    metadata:
      labels:
        app: ai-sec-gateway
    spec:
      containers:
        - name: gateway
          image: your-registry/ai-sec-gateway:latest
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_DATA_REDIS_HOST
              value: redis.default.svc.cluster.local
            - name: SPRING_KAFKA_BOOTSTRAP_SERVERS
              value: kafka.default.svc.cluster.local:9092
            - name: FIREWALL_THREAT-ENGINE-URL
              value: http://threat-engine.default.svc.cluster.local:8000
            - name: FIREWALL_JWT-SECRET
              valueFrom:
                secretKeyRef:
                  name: gateway-secrets
                  key: jwt-secret
          resources:
            requests: { cpu: "250m", memory: "384Mi" }
            limits: { cpu: "1", memory: "768Mi" }
          readinessProbe:
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 15
            periodSeconds: 10
          livenessProbe:
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 30
            periodSeconds: 20
      ---
apiVersion: v1
kind: Service
metadata:
  name: ai-sec-gateway
spec:
  selector:
    app: ai-sec-gateway
```

```

    ports:
      - port: 80
        targetPort: 8080
        type: LoadBalancer
    ---
    apiVersion: autoscaling/v2
    kind: HorizontalPodAutoscaler
    metadata:
      name: ai-sec-gateway-hpa
    spec:
      scaleTargetRef:
        apiVersion: apps/v1
        kind: Deployment
        name: ai-sec-gateway
      minReplicas: 3
      maxReplicas: 12
      metrics:
        - type: Resource
          resource:
            name: cpu
            target:
              type: Utilization
              averageUtilization: 65

```

16.2 What's Genuinely Missing for Production

Gap	Why it matters	How you'd close it
No mTLS between internal services	Anyone on the Docker network can currently talk to Redis/Kafka/ES directly, bypassing the gateway entirely	Service mesh (Istio/Linkerd) or manual mTLS via a cert-manager + sidecar setup
Secrets in plaintext env vars	<code>firewall.jwt-secret</code> sits in <code>application.yml</code> /Compose env	HashiCorp Vault, AWS Secrets Manager, or K8s Secrets + external-secrets-operator
No GeoIP integration	<code>geo_velocity_kmph</code> is stubbed at 0	MaxMind GeoIP2 or a managed IP-intelligence API, with its own latency budget
Single-node Kafka/Elasticsearch	No fault tolerance, no real throughput headroom	3+ broker Kafka cluster with replication factor 3; multi-node ES cluster with proper shard allocation
Model retraining is manual	Attack patterns drift; a static model decays	Scheduled retraining job against real labeled traffic, versioned model registry (MLflow), shadow-deploy new models before promoting
No model monitoring	You won't notice the model degrading until damage is done	Track score distribution drift, feature drift (e.g. population stability index), and false-positive rate from analyst feedback
<code>distinct_endpoints_60s</code> stubbed on sync path	Weakens sync-path detection of scanning-style behavior	Redis <code>SADD</code> with a 60s TTL per identity, read as <code>SCARD</code>
No WAF/OPA-style declarative rule layer	Ad-hoc thresholds in YAML don't scale past a few dozen rules	Open Policy Agent (OPA) sidecar for declarative, versioned policy — a natural pairing with this architecture
No human-in-the-loop review queue	<code>CHALLENGE</code> decisions currently just get a header	A lightweight internal tool for analysts to review <code>CHALLENGE</code> -tier events and feed labels back into retraining

16.3 Suggested Build Order If You Want to Extend This

1. Wire up real GeoIP and a Redis-backed `distinct_endpoints_60s`.
2. Add an OPA sidecar and move the high-risk-endpoint list and numeric baselines into declarative policy.
3. Stand up a 3-broker Kafka cluster and a 3-node Elasticsearch cluster (even just in Docker Compose, to practice the operational model).
4. Build the model-monitoring dashboard (score distribution over time, per-feature drift) before you build automated retraining — you need to see the problem before you can automate the fix.
5. Only then automate retraining, with a shadow-deploy step that compares the new model's decisions against the live model's for a day before promoting it.

Building even two or three of these is worth more on a resume than the base project alone — it shows you know the difference between “runs on my laptop” and “runs in production,” which is precisely the judgment senior roles screen for.

17. Known Limitations — Read This Before You Claim “Production-Ready”

Be precise about these in an interview; overselling a portfolio project is a much bigger red flag than the limitations themselves.

1. **The model is trained on synthetic data with clean class separation.** The perfect classification report in Section 6.3 reflects the separability of hand-tuned statistical distributions, not real-world discriminative power. Real traffic overlaps far more; expect a real deployment's ROC-AUC to look nothing like 1.0, and to require iteration on features and thresholds against real, labeled data.
2. **numeric-baselines are hand-configured per field**, not learned. Section 16.2 covers what a calibrated version would look like.
3. **The synchronous scoring path sees a reduced feature set** relative to the asynchronous path, by design (Section 7.9) — but that means a sufficiently fast, sufficiently novel business-logic attack that doesn't trip the rate threshold could complete before the async path catches up. This is a real, named trade-off, not an oversight.
4. **No mTLS, no secrets manager, no multi-node data stores** — see 16.2 in full.
5. **The circuit breaker and timeout values (25ms, 50-call sliding window, 50% failure threshold) are reasonable starting points, not measured production tunings.** Section 14.4 shows you how to actually measure and justify your own numbers instead of keeping mine.

If you present this project and someone probes on any of these, the strongest possible answer is naming the limitation before they do, explaining why it exists, and describing the concrete next step — that is a more senior signal than a project with no visible seams at all.

18. Making This Resume, Portfolio, and Interview Ready

Building the system is half the value. The other half is being able to talk about it in 30 seconds, in 3 minutes, and under adversarial follow-up questioning — those are three different skills, and this section drills all three.

18.1 Resume Bullets

Use quantified, verb-first language. Pick 2–3 — don't use all of them, that reads as padding.

- Designed and built a distributed, zero-trust API gateway (Spring Boot, Kafka, Redis) that scores live traffic against an ensemble ML model (Isolation Forest + XGBoost) to detect credential stuffing and business-logic abuse in real time.
- Architected a dual-speed detection pipeline — a sub-25ms synchronous path backed by Redis, and an asynchronous Kafka-driven path for full-fidelity ML scoring — reducing worst-case added latency on benign traffic to near zero.
- Implemented reactive, non-blocking request inspection in Spring WebFlux, including safe request-body buffering and replay for payload analysis without breaking downstream proxying.
- Trained and evaluated an ensemble anomaly-detection model combining unsupervised (Isolation Forest) and supervised (XGBoost) approaches, with a calibrated 0–1 risk score driving allow/challenge/block decisions.
- Built a full audit and observability pipeline (Kafka -> Elasticsearch -> Kibana) indexing 100% of gateway decisions for security analyst querying, backed by an ILM-managed retention policy.
- Containerized a 6-service distributed system with Docker Compose, health-check-gated startup ordering, a GitHub Actions CI pipeline, and a sample Kubernetes deployment manifest with HPA.

18.2 The 30-Second Elevator Pitch

"I built a zero-trust API gateway that sits in front of a microservices backend and scores every request for two kinds of attacks signature-based firewalls can't see: credential stuffing and business-logic abuse — like a tampered price or quantity. It uses an ensemble of an unsupervised and a supervised ML model, and it's designed around a real latency budget: a fast synchronous path using Redis for immediate enforcement, and an async Kafka pipeline that does the heavier ML work and feeds a blacklist back into the fast path. Everything's containerized, there's a CI pipeline, and I wrote attack simulation scripts that prove it actually blocks what it claims to."

18.3 STAR-Format Stories for Behavioral Rounds

"Tell me about a challenging distributed systems problem you solved." *Situation:* needed to inspect request payloads for business-logic anomalies inside a reactive gateway. *Task:* Spring WebFlux request bodies are single-read reactive streams — reading the body for analysis would normally consume it, breaking the proxy to the backend. *Action:* implemented `DataBufferUtils.join` to buffer the body once, then wrapped the request in a `ServerHttpRequestDecorator` that replays the cached bytes for the downstream call. *Result:* the gateway can inspect JSON payloads for numeric-field anomalies without breaking transparent proxying — and I can explain exactly why naive approaches (reading the body twice, or not re-wrapping the request) fail.

"Tell me about a time you had to make a trade-off between two good options." *Situation:* the Threat Engine call sits on the gateway's request path. *Task:* decide what happens if that call is slow or the service is down. *Action:* chose fail-open with a bounded timeout and a circuit breaker, specifically for browsing-type endpoints, reasoning that blocking real users on a dependency hiccup is worse than a few hundred milliseconds of reduced detection (which the async path recovers). *Result:* documented this explicitly as a per-endpoint policy rather than a global rule, since a wire-transfer endpoint should make the opposite trade-off — and can walk through exactly how I'd implement that difference.

"Tell me about a time you used ML in a production-shaped system, not just a notebook." *Situation:* needed real-time risk scoring inside an API gateway. *Task:* the model has to run inside a latency budget and degrade gracefully. *Action:* separated feature engineering into a single shared module used by both training and serving to avoid train/serve skew, combined an unsupervised and supervised model into a calibrated ensemble score, and split scoring into a cheap synchronous path and an expensive asynchronous one. *Result:* can speak concretely about calibration, thresholding, and the specific trade-off between model complexity and latency — not just "I trained a model."

18.4 Questions to Expect — and How to Answer Them

Likely question	What a strong answer touches
"Why not just rate-limit at the load balancer?"	Rate limiting alone can't see semantic payload anomalies (business-logic abuse) and is a blunt instrument against distributed/slow attacks — explain the two attack classes and why each needs different detection.

Likely question	What a strong answer touches
"How do you know your model actually works?"	Be upfront about synthetic training data (Section 17), then pivot to what you validated: unit tests on feature extraction and decision boundaries, and end-to-end attack simulations with real captured output.
"What happens if Kafka goes down?"	Gateway keeps functioning — Redis blocklist checks are independent of Kafka, and the publish call is fire-and-forget with error handling. You lose the async learning loop and audit trail until Kafka recovers, not enforcement.
"Why Isolation Forest AND XGBoost instead of just a neural net?"	Talk about the unsupervised/supervised split (Section 3.4) and interpretability/operational simplicity — tree ensembles are cheaper to run, easier to explain to a security analyst, and don't need GPU infrastructure.
"How would you scale this to 100x the traffic?"	Kafka partitions, horizontal gateway replicas (stateless besides Redis), Redis Cluster, multi-node Elasticsearch — and specifically explain why the gateway can scale horizontally with zero coordination (Section 16.2).
"What's the biggest weakness in this design?"	Pick a real one from Section 17 and describe the fix — this is the single highest-signal answer you can give in a system-design interview.

18.5 GitHub Repository Presentation

The `README.md` built alongside this guide (and included in full in the accompanying repository) is written the way hiring managers actually skim READMEs: a one-line pitch, an architecture image above the fold, a `docker compose up` quickstart that actually works, and an explicit "known limitations" section. That last part is unusual and it is a deliberate signal — reviewers see hundreds of portfolio projects that claim to be production-ready; one that precisely states what it isn't stands out.

Suggested repo name: `ai-api-security-firewall` or `zero-trust-api-gateway`. Pin it on your GitHub profile. Add the architecture diagram as the repo's social preview image (Settings -> General -> Social preview) so it renders when shared as a link.

Appendix A — Full Repository File Tree

```

ai-api-security-firewall/
  .gitignore
  LICENSE
  README.md
  docker-compose.yml
  .github/
    workflows/
      ci.yml
  audit-indexer/
    Dockerfile
    indexer.py
    requirements.txt
  docs/
    architecture.png
    request_flow.png
  elasticsearch/
    audit-events-template.json
    ilm-policy.json
  gateway/
    Dockerfile
    pom.xml
    src/
      main/
        java/
          com/
            aisecfirewall/
              gateway/
                GatewayApplication.java
              client/
                ThreatEngineClient.java
              config/
                GatewayConfig.java
                SecurityProperties.java
              dto/
                RequestSignalDTO.java
                ScoreResponseDTO.java
              filter/
                JwtAuthValidator.java
                ZeroTrustFilter.java
              service/
                KafkaRequestPublisher.java
                RedisStateService.java
            resources/
              application.yml
      test/
        java/
          com/
            aisecfirewall/
              gateway/
                filter/
                  JwtAuthValidatorTest.java
  k8s/
    gateway-deployment.yaml
  mock-services/
    auth-service/
      Dockerfile
      main.py
      requirements.txt
    order-service/
      Dockerfile
      main.py
      requirements.txt
  scripts/
    locustfile.py
    query_audit_trail.py
    setup_infra.sh
    simulate_business_logic_abuse.py
    simulate_credential_stuffing.py
  threat-engine/
    Dockerfile
    requirements-dev.txt

```

```
requirements.txt
app/
  __init__.py
  features.py
  kafka_consumer.py
  main.py
  model.py
models/
tests/
  test_features.py
  test_model.py
train/
  train_model.py
```

Appendix B — Command Cheat Sheet

Local development (fastest iteration loop, no Docker)

```
cd threat-engine
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python train/train_model.py
pytest -q
uvicorn app.main:app --reload --port 8000
```

Gateway, standalone

```
cd gateway
mvn spring-boot:run
```

Full stack

```
docker compose up -d --build      # build and start everything
docker compose ps                 # check health status
docker compose logs -f gateway    # tail one service's logs
./scripts/setup_infra.sh          # provision Kafka topics + ES template/ILM
docker compose down -v           # stop and wipe volumes (fresh start)
```

Attack simulations

```
python scripts/simulate_credential_stuffing.py --target http://localhost:8080
python scripts/simulate_business_logic_abuse.py --target http://localhost:8080 --token <JWT>
python scripts/query_audit_trail.py --es http://localhost:9200
```

Load testing

```
pip install locust
locust -f scripts/locustfile.py --host http://localhost:8080 \
  --users 50 --spawn-rate 10 --run-time 1m --headless
```

Kafka introspection

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh --list --bootstrap-server localhost:9092
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic api.audit.events --from-beginning
```

Redis introspection

```
docker exec -it redis redis-cli
> KEYS blocklist:*
> TTL blocklist:203.0.113.7
> GET failstreak:demo_user
```

Elasticsearch / Kibana

```
curl http://localhost:9200/_cat/indices?v
curl "http://localhost:9200/api-audit-events-*/_search?size=5&sort=timestamp:desc"
# Kibana UI: http://localhost:5601
```

This guide and the accompanying repository were built and verified section by section — the ML pipeline was trained and queried live, the mock services were smoke-tested, and every embedded code listing is copied directly from the working project rather than retyped. Treat Section 17 as the honest scope of what “verified” means here: the distributed orchestration is correct by careful construction and individual-component testing, not by a full docker-compose run in the environment this guide was written in. Run it, break it, extend it — and good luck.